

JJ DAI · Роадмэп до запуска сети · r6.9.13

JAI · DECENTRALIZED ARTIFICIAL INTELLIGENCE

Единый документ · включает r6.7 целиком + шестой сквозной трек «Отложенная верификация и оффлайн-режим узла» + Agent Alpha как цель фазы + седьмой сквозной концерт «Происхождение исполняемого кода» · 26 августа 2026 · jj-dai.org

Документ самодостаточен: чтение r6.7 не требуется. От билда v0.6.8 к testnet-0, Mainnet Core и Mainnet Public. Учтены все раунды ревью Агента-Аудитора, включая ADR-014 (с поправкой A-1), ADR-015 (Accepted), ADR-016 rev 2 (Proposed), ADR-017 с поправкой A-1 (Accepted), ADR-018 rev 2 (Accepted), **ADR-019 rev 3.1 (Accepted)**, **ADR-020 (база rev 4.1 Accepted · поправки rev 4.3.1 Proposed)**, **ADR-021 rev 4 (Proposed)** и **ADR-022 rev 2.4 (Accepted 5 сентября 2026)**.

r6.9.13. Ревизия по аудиту rescut7. **Правок по существу плана нет** — счётчик прежний, 308, потому что оба сценария вошли в существующую REL-24, а не в новую проверку. Аудит подтвердил большинство исправлений rescut7 и нашёл два места, где недопустимое событие всё ещё попадало в append-only цепь: `revoke_release()` проверяла подпись и позицию, но не сверяла реестр с живой проекцией жизненного цикла — отозванный ключ с устаревшим реестром (`revoked: None`) записывал `RELEASE_REVOKED`; и `publish()` записывала публикацию в цепь, которая уже не верифицируется, — обнаружение работало, но на одну запись позже, чем позволяет история без правок. Оба закрыты одним правилом: **целостность цепи и полнота проекции жизненного цикла — предусловия записи**, проверяемые под тем же удержанием, что и сама запись, в `publish()` и в `revoke_release()` одинаково. Отказ ничего не пишет ни в память, ни в журнал, и проверка утверждает оба размера. Содержание r6.9.12 сохранено целиком.

r6.9.12. Ревизия по аудиту rescut6. **Правок по существу плана нет** — механизм rev 2.4 принят аудитом как нужный и сделанный в правильном месте, но закрытие двух позиций D6 признано преждевременным: воспроизведены пять дефектов в `jjdai/release.py`, все подтверждены и закрыты в rescut7. `revoke_release` подписывала и записывала, не проверяя подпись под зарегистрированным ключом — недействительный отзыв попадал в append-only цепь и по fail-closed правилу блокировал выпуск навсегда; `check_lifecycle_witnessed` проверяла только позиции, которые реестр предъявил, и записанный отзыв ключа скрывался значением `revoked: None` — теперь реестр сверяется с проекцией цепи, и утаённый переход отказывает; `_check_not_revoked` не сверяла `statement_hash` подписанного тела с целевым выпуском, и подпись об отзыве В принималась как отзыв А — теперь три значения обязаны совпасть; созданный до отзыва `authorizer` продолжал разрешать — теперь он перепроверяет снимок публикации против живой цепи на каждом вызове, сохраняя неизменяемость того, ЧТО разрешено; проверки жизненного цикла и отзыва в `publish()` шли до захвата блокировки, и отзыв вклинивался между проверкой и записью — теперь они под тем же удержанием, что и запись, а `revoke_release` судит действительность ключа на позиции, которую запись фактически займёт. Плюс P1: «64-hex» в `witness.append` проверял длину, не алфавит. События `DEBT_CLOSED` rescut6 остаются в ledger — он append-only и вида «геореп» не имеет; что доказательства на момент закрытия были недостаточны, записано здесь и в `CHANGELOG`, а не стёрто. Счётчик 307 → **308**. Содержание r6.9.11 сохранено целиком.

r6.9.11. Ревизия по аудиту rescut5 и по **ADR-022 rev 2.4**. Вторая правка по существу решения. Аудит снял HOLD с rescut5 и поправил одну фразу HANDOFF — «следующий шаг лежит вне дерева» — как преждевременную: две позиции долга, `release-key-lifecycle-witnessing` и `release-revocation-by-name`, были программно-нормативными, и церемония на оборудовании их не закрыла бы. Он был прав, и порядок был обратный названному: сначала решение, код и acceptance, затем сборка и выпуск. Решение — rev 2.4, принятая **до первой эмиссии**, потому что после первой `RELEASE_ATTESTED` добавление вида записи в хэш-сцепленную цепь есть миграция. Шестое окно получает три эмитируемых вида записей — `KEY_ACTIVATED`, `KEY_REVOKED`, `RELEASE_REVOKED`, — два префикса прообраза, один домен подписи и одну схему. `check_lifecycle_witnessed` в третьей версии делает то, что обещал её докстринг в первой: позиция реестра разрешается в запись объявленного вида, называющую этот ключ и этот переход. Именованный отзыв выпуска, несённый в v0.6.9 как функция, читавшая поле, которого никто не пишет, вернущийся настоящим: подписанное заявление плюс запись, аттестация нетронута, отозванный выпуск отказывается на всех потребляющих путях, неразрешимый отзыв не снимается. Обе позиции закрыты событиями `DEBT_CLOSED` со ссылкой на проверки; граница публикации в ledger чиста. Открытых позиций восемь. Счётчик 306 → **307**. Содержание r6.9.10 сохранено целиком.

r6.9.10. Ревизия по аудиту rescut3. **Правок по существу плана нет** — словарь ADR-022 rev 2.3 досинхронизирован в действующих требованиях, счётчик приведён к **306**. Аудит принял доказательную модель rev 2.3 по существу и нашёл три блокера на границах, которые проверяют её: авторизация тулсета получалась и без свидетельства пересборки — `toolset_authorizer` вызывал диагностическую `verify_publication` с любым контекстом, а отсутствующий `evidence_reader` означал пропуск проверки; выданное разрешение менялось вслед за словарём — `callback` держал ссылку на `statement`, а не проверенные значения; очистка окружения начиналась слишком поздно — `python -m venv и pip install --prefix` наследовали оболочку, и окружение формировалось непиненным путём поиска на шаг раньше чистой сборки. Все три закрыты, плюс четыре P1: `validate_manifest(doc, raw=...)` допускал два разных объекта под одной подписью; ассемблер `statement` включал в артефакты собственный предыдущий `output`; в этом документе перечень сепараторов шестого окна, таблица окон, требование перед резкой и объём Audit 0 называли rev 2.2 как действующую; HANDOFF переоценивал закрытие P1. Исправлено ровно это. **Открытых позиций долга десять, а не восемь**, как утверждал HANDOFF rescut3: `cla` и `t-toolset` открыты и блокируют свои границы. Содержание r6.9.9 сохранено целиком.

r6.9.9. Ревизия по аудиту rescut2 и по **ADR-022 rev 2.3**. Первая с r6.9 правка **по существу решения**, а не по состоянию: аудит вынес HOLD и показал, что закрытие P0-3 предыдущего реката было подменой того же класса, которую этот дроп ловит четвёртый раунд подряд. Свидетельством независимой пересборки был объявлен `ReleaseStatement` пересборщика — и им оказался **сам основной statement**: объект свидетельствовал о себе, сравнение полей проходило тривиально, потому что сравнивался документ с самим собой. Разрешимость была доказана, независимость — нет. Вторая находка того же корня: `reproducibility_score` был самообъявленным значением `enum`, и тег с `cross-host-class` ставился при полном отсутствии сведений о хостах обеих сторон — код принимал усиленное доказательство, которого свидетельство не предъявляло. Оба закрываются **только поправкой к ADR**, потому что нужны факты, которых ни одна замороженная схема не несла; поправка принята **до первой эмиссии** — ни тега, ни релизных ключей, ни одной записи `RELEASE_ATTESTED` в дереве нет, а после церемонии та же правка была бы миграцией. Вводится `jjdai.rebuild-evidence/v1` с собственным доменом подписи, каждое одобрение несёт `build_run_id` и `host_class`, а область воспроизводимости **вычисляется** из двух прогонов и сверяется с записанной. Плюс закрыты четыре остальных P0 аудита: резолвер авторизации тулсета доведён до двери, которую он охраняет (`kernel/isolation.py` его не передавал, и построенная проверка снова стояла рядом с границей); адрес манифеста сведён к одному — `sha256` файла, потому что ассемблер писал байты файла, а загрузчик считал JCS разобранного документа, и две стороны адресовали разные объекты; долг жизненного цикла ключей перенесён с границы тега на **публикацию**, потому что `RELEASE_ATTESTED`, построенная на редактируемых позициях, уже годилась как время авторизации L2 при запрещённом теге; окружение сборки строится по `allowlist`, а не наследуется — внешний `PYTHONPATH` доходил до `python -m build` и мог подменить сам проверенный frontend. Счётчик 300 → **304**. Содержание r6.9.8 сохранено целиком.

г6.9.8. Ревизия по аудиту rescut1. **Правок по существу плана нет** — счётчик приведён к фактическим **300** после rescut2 по пяти P0, и названы две новые позиции долга. Аудит вынес HOLD и нашёл, в частности, что REL-16 подтверждал свойство, которого код не реализует: функция `check_lifecycle_witnessed` обещала в докстринге читать прообраз записи, делала поиск по индексу и запись выбрасывала, не вызываясь ни из одного production-пути, а KEY_ACTIVATED в словаре витнесса никогда не стоял — сама проверка добавляла шесть записей INFER и принимала одну из них за активацию ключа. **Ложный green хуже отсутствующей проверки, потому что отсутствующая видна.** Плюс закрыты: самообъявленная позиция авторизации тулсета (теперь резолвится из публикации, назвавшей манифест), неразрешаемое свидетельство пересборки (теперь это собственный ReleaseStatement второго сборщика — новой схемы не заводится, шестое окно заморожено), гейт тега, не читавший канонический ledger долга (при семи открытых позициях он выдавал ACCEPTED), и подготовка сборки, устанавливавшая пакеты в текущий интерпретатор вместо одноразового окружения. Что доказать в дереве нельзя — названо долгом, а не изображено: `release-key-lifecycle-witnessing` и `build-egress-enforcement`, обе блокируют смысл релизного тега. Отдельно исправлен документарный дефект, который аудит нашёл в самом этом файле: 219/219 в нормативном абзаце о нумерации рядом с верным 295/295 в статусной формуле — тот самый класс «правильный текст рядом с устаревшим», от которого документ предостерегает в своих сквозных принципах. SYNC-8 его не видела, потому что искала одну формулировку N/M recorded; теперь она проверяет всякую пару в нормативной прозе, а журнал ревизий и таблицу дропов исключает как историю. Содержание г6.9.7 сохранено целиком.

г6.9.7. Ревизия по аудиту резки v0.6.9. **Правок по существу плана нет** — счётчик приведён к фактическим **295** после rescut'a по девяти P0. Аудит нашёл, в частности, что WitnessChain.append с v0.5.3 нёс комментарий «guards append» и блокировку ни разу не брал: два потока получали индекс 0 и цепь переставала верифицироваться. Это дефект самой цепи, а не релизного кода, — релизный код лишь сделал его видимым. Плюс закрыты: публикация с неполным контекстом, красная приёмка, непроверяемая таблица окружения, prefix-исключения дайджеста без привязки, историческая действительность тулсет-ключа и длина артефакта. Именованный отзыв выпуска записан долгом, а не симитирован: ему нужен вид записи, которого нет в шестом окне резерва. Содержание г6.9.6 сохранено целиком.

г6.9.6. Ревизия по резке v0.6.9. **Первая с г6.9 правка по существу состояния, а не по редактуре.** Дроп v0.6.9 code-complete: построена вся цепь ADR-022 от D2 до D14 плюс T-TOOLSET, счётчик acceptance вырос с 219 до **292**. Статусная формула переписана на v0.6.9 в обоих местах, где она стоит. Долг закрыт двумя позициями из семи — `readme-outside-tree-digest` и `file-modes-and-symlink-targets`, каждая со ссылкой на проверку; пять остаются открытыми, и это записано намеренно: построенный механизм не есть уплаченный долг. Колёс backend'a, ключей релиза, собранного bundle и скомпилированных модулей в дереве нет, и дерево отказывает без них — `BUILD_PIN_MISSING`, `RECIPE_PLACHOLDER`, `REL_UNKNOWN_KEY`, `MANIFEST_NO_RESOLVER`. Тер не ставится: он требует хоста с тулчейном, второй машины того же класса и церемонии ключа на двух устройствах — позиций гейта Ф0, а не сборки. Плюс `jjdai.__version__` переведён на 0.6.9 по тому же правилу, по которому он стоял на 0.6.8 у нетегированного code-complete дропа. Содержание г6.9.5 сохранено целиком.

г6.9.5. Ревизия по четвёртому аудиту pre-release. **Правок по существу плана нет** — три редакционных места, два из них о статусе ADR-020. Из footer убраны вторая, дублирующая ссылка на ADR-020 и обе хвостовые пометки «(Accepted, ...)»: г6.9.4 объявила, что лишнее «(Accepted)» после смешанного статуса снято, и это было верно для первой страницы и неверно для последней — тот самый класс дефекта «правильный текст рядом с устаревшим», от которого документ предостерегает в своих сквозных принципах. Действующая формула одна и стоит везде: база rev 4.1 Accepted, набор поправок rev 4.3.1 Proposed. Вне роадмэпа тем же катом исправлены §3 ADR-020, где фраза «первые два... третий» описывала перечень из трёх видов записей, тогда как после rev 4.3.1 в нём четыре вида и отдельная строка схемы, и дубль строки SYNC-8 в докстринге проверок. Содержание г6.9.4 сохранено целиком.

г6.9.4. Ревизия по третьему аудиту pre-release. **ADR-022 rev 2.2 переведён в Accepted** — условие резки v0.6.9 выполнено, предгенезисный резерв больше не стоит на Proposed-решении. Счётчик приведён к фактическим **219** (206 + 9 + 2 + 2: LEDGER-1..3, SYNC-7, SYNC-7-MUT, SYNC-8), и **введена SYNC-8**, которая сверяет число в роадмэпе с записанным прогоном и с порождёнными поверхностями: раньше эту связку не проверял никто — R-ACCEPT сверяет бейдж с прогоном, `check_docs_drift` сверяет поверхности с их источником, и ни одна из двух не читает роадмэп. SYNC-8 сверяет **только итог**, а не число пройденных: число пройденных в нормативной прозе полицируется бейджем, который рендерится из прогона и потому не может лгать, а требовать его здесь значило бы повторить круговую зависимость, которую аудит v0.6.7 уже разбирал. Снято лишнее «(Accepted)» после смешанного статуса ADR-020. Содержание г6.9.3 сохранено целиком.

г6.9.3. Ревизия по второму аудиту pre-release. **Правок по существу плана нет** — документ противоречил сам себе о статусе ADR-020 между первой и последней страницей. Первая страница несла «ADR-020 (база rev 4.1 Accepted · поправки rev 4.3.1 Proposed)», последняя — «база rev 4.1 Accepted, набор поправок Proposed»; действует вторая формула, и теперь она стоит везде. **Арифметика 217 исправлена:** документ объяснял число как 206 + девять, что даёт 215, и молчал о двух проверках, добавленных доработкой (LEDGER-1..3 и SYNC-7 вместе с SYNC-7-MUT). **SYNC-7 переписана по реестру:** первая реализация была эвристической, и mutation-тест ревьюера прошёл сквозь неё — теперь источник правды `drop_plan` в `architecture_status.json`, каждый ADR несёт нормативную строку Drop: , а проверка возит **собственные mutation-тесты**, доказывающие, что она вообще способна покраснеть. Плюс снята редакционная фраза «ссылки на rev 4.1 заменены на rev 4.3», где обе половины назывались одинаково. Содержание г6.9.2 сохранено целиком.

г6.9.2. Ревизия по аудиту pre-release. **Правок по существу плана нет** — четыре места, где документ утверждал о дереве то, чего аудит в дереве не нашёл, и одно, где он ссылался на непринятую ревизию. Счётчик acceptance приведён к фактическому: в дереве **217** проверок (доработка добавила LEDGER-1..3 и SYNC-7 к 215 первого кат), и все поверхности несут одно число вместо трёх разных. Утверждение «drift чист» снято как непроверяемое на момент чтения: оно описывало прогон, а не состояние, и держалось после того, как состояние менялось. Ссылки на **ADR-020: база rev 4.1 Accepted, набор поправок rev 4.3.1 Proposed** заменены на **rev 4.3**, которая понижена до Proposed — rev 4.2 объявила себя синхронизацией и при этом ввела новый предгенезисный вид записи, что по правилам проекта есть архитектурное изменение. **SYNC-7 реализована**, а не объявлена должной: ни один ADR не вправе называть номер дропа, расходящийся с таблицей дропов роадмэпа. Содержание г6.9.1 сохранено целиком.

г6.9.1. Ревизия по ревью связи «г6.9 + ADR-020». Правок по существу решения нет — шесть мест, где документ после перестановки дропов заново открыл то, что ADR уже закрывал. **Правило заморозки разделено по классам:** «в Ф0 замораживаются только имена, схемы — в Ф2» относилось к трекам V и VI, которые в Ф0 ничего не эмитируют; схемы ADR-022 и ADR-020 эмитируются внутри Ф0 и потому фиксируются целиком до первой эмиссии. **SELF_CHECKED в Ф0 не эмитируется:** custody runtime стоит в Ф2, проверяется только рендеринг бейджа на синтетической фикстуре, которая в WitnessChain не попадает. **Гейт Ф0 перестал требовать полных сессий G-GUI на трёх узлах:** это гейт Ф1, а в Ф0 требование смешивало обратно три набора evidence, которые г6.8.1 развела. **Acceptance ADR-020 привязан целиком**, включая N-1...N-18 и persist-before-expose. **Приватная часть обратной связи и отдельный keystore хранителя внесены в объём и в гейт.** Старая формула о греореп убрана — она сосуществовала с новым определением риге и противоречила ему. Плюс: заголовок «Три окна» переименован, окна расставлены по очерёдности дропов. Содержание г6.9 сохранено целиком.

r6.9. Ревизия по решению владельца о порядке дропов и по ADR-022. **Одно решение по существу плана и его следствия.** Порядок дропов меняется: **v0.6.9 несёт T-TOOLSET и supply chain, v0.6.10 несёт Agent Alpha**, v0.6.11 не меняется. Agent Alpha **уезжает** в следующий дроп, а не теснится рядом с тулсетом — критический путь Alpha не расширяется, он сдвигается. Основание — устойчивость расписания: дроп Alpha зависит от настоящей модели, двух классов хоста, трёх наборов evidence и церемонии ключа хранителя, а инфраструктурный дроп зависит только от дерева и двух устройств; ставить дроп с открытым краем перед релизным конвейером значит держать конвейер заложником. **Цена названа:** первый ручной контакт с JaiguruDev откладывается на дроп, и возращение «сетью нельзя пощупать руками» перестановкой не снимается, а отодвигается. **Условие возврата:** если v0.6.9 не порезан до 16 сентября 2026, порядок возвращается к r6.8.5, и это фиксируется переносом невыполненного, а не новой очерёдностью. **Two-person release approval отменён** (ADR-022 D2) — требование не имело под собой ADR и при одном принципе невыполнимо по существу; заменено two-key контролем, описанным числами. **Долг стал событийным ledger**, позиция не удаляется, а получает DEBT_CANCELLED со ссылкой на решение либо DEBT_CLOSED со ссылкой на проверку. Формулировка о production-руке исправлена: рука закрывается в v0.6.9, но её класс эффекта — `rule`. Содержание r6.8.5 сохранено целиком.

r6.8.5. Ревизия по аудиту rescu2. Правок по существу плана нет — три уточнения о состоянии и границах. **Норма .gitattributes** переписана: требование «первым коммитом, до первого клона» описывало репозиторий, которого ещё нет, и для существующего невыполнимо ретроактивно; действующая норма привязана к коммиту под тегом. **Границы пиннинга названы:** python-зависимости тестов закреплены по хэшу, экшены workflow — по коммиту, build backend — только по версии, и последнее объявлено открытым, потому что PEP 517 не даёт места для хэша. **Свежесть PDF стала проверяемой:** sha256 markdown штампуется внутрь PDF, SYNC-6 сверяет — в rescu2 PDF роадмэпа нёс 211/211 рядом с markdown на 213/213, и ни одна проверка дерева этого не видела, потому что `check_docs_drift` проверяет только поверхности, порождаемые из `architecture_status.json`. Содержание r6.8.4 сохранено целиком.

r6.8.4. Ревизия по аудиту rescu1. Правит четыре места, где документ продолжал описывать состояние дерева ДО доработки, притом что его собственное резюме описывало состояние ПОСЛЕ: § «Байтовая точность дерева» утверждал отсутствие `.gitattributes`; критерии выхода Ф0 держали в релизном долге CLA, SBOM и пиннинг; аннотация предполётного тега требовала писать «SBOM отсутствует»; § «Где мы сейчас» перечислял отсутствующими четыре позиции, из которых реально отсутствует одна. Это тот самый дефект, от которого документ предостерегает в своих сквозных принципах: **правильный текст рядом с устаревшей таблицей хуже одного неверного текста** — здесь он появился в самой ревизии, которая это правило и цитирует. Плюс: «Три условия постановки» переименованы в четыре (четвёртое добавлено в r6.8.3, а заголовок остался), и релизный долг переписан списком того, что действительно открыто. Содержание r6.8.3 сохранено целиком.

r6.8.3. Ревизия по аудиту связи «билд v0.6.8 + роадмэп r6.8.2». Закрывает один P0: § «Четвёртое окно» несло резерв **ADR-020 rev 2** — два доменных сепаратора из пяти, без оснований удаления тела, — хотя выше документ объявлял rev 4.1 Accepted; там же § «Что остаётся открытым» требовал принять rev 2 до резки v0.6.9. Резерв заменён дословным перечнем из §3 ADR-020 (база rev 4.1 Accepted · поправки rev 4.3.1 Proposed), и в объём v0.6.9 включён **весь согласованный набор изменений**, ни одна часть которого не действует отдельно. Плюс: статус v0.6.8 приведён к одной формуле в четырёх местах; CLA убрана из причин отсутствия тега (она блокирует приём вклада, а не тег); путь записанного прогона исправлен на фактический; в условия предполётного тега добавлена синхронность нормативных документов с деревом; SBOM, пиннинг и `.gitattributes` перенесены из v0.6.10 в доработку v0.6.8. Содержание r6.8.2 сохранено целиком.

r6.8.2. Добавляет предполётные теги в правила нумерации, называет перенос невыполненного содержания v0.6.7 переносом, вносит байтовую точность дерева в supply chain и обновляет статусы ADR-017 A-1, ADR-020 и ADR-021. Содержание r6.8.1 сохранено целиком.

r6.8 → r6.8.1. Точечная ревизия по ревью ADR-020. Блоки A-ALPHA и G-GUI в r6.8 были написаны до того, как онтология Agent Alpha была разобрана, и содержали четыре ошибки: смешение харнесса, профиля, существа и класса хоста; прямой переход разметки хранителя в кандидаты канареек; роль `admin` вместо узкой `guardian_ui`; и инвариант «новых эндпоинтов нет», несовместимый с тем, что подписанной разметки исхода в демоне не существует. Плюс r6.8 противоречил сам себе о статусе v0.6.8. Шестнадцать правок внесены; всё остальное содержание r6.8 сохранено без изменений.

Изменения r6.7 → r6.8

Область	Было в r6.7	Стало в r6.8
Сквозной трек VI	отсутствует	Отложенная верификация и оффлайн-режим узла: узел входит в custody автоматически при недоступности панели, решение получает второе состояние допуска <code>SELF CHECKED</code> , рука ограничена локально-обратимым, память под отложенной верификацией остаётся полностью доступной существу и различается только доказательным статусом. Основание — ADR-019 rev 3.1. Резерв в Ф0 (выполнено, v0.6.8), вертикаль в Ф0, спецификация в Ф2, сетевая корроборация в Ф3
Agent Alpha	понятия нет	названная цель фазы Ф0: узел, с которым хранитель может поговорить руками и разметить исход. Содержание — G-GUI плюс прогон живой вертикали на целевом хосте. Носитель — v0.6.9
Таблица дропов Ф0	v0.6.7 и v0.6.8 «план»	кодový объём v0.6.8 выполнен, acceptance записан; дроп не закрыт и не выпущен. Перенумерация: v0.6.9 Agent Alpha, v0.6.10 T-TOOLSET вместе с supply chain, v0.6.11 каталог evidence-ID
Резерв трека V	«до genesis (v0.6.8)» в критериях выхода	выполнено. Вместе с ним выполнен резерв ADR-019, который §4 того же ADR назначает этому же дропу
Разведение Viveka / KriyaGate	сериализуемые значения в v0.6.8, переименование модуля в Ф2	сериализуемые значения выполнены; <code>ORGAN_BINDINGS</code> несёт обе строки, эмитируется <code>kriya_gate</code> . Переименование модуля остаётся в Ф2
Объём Audit 0	ADR-015, ADR-016 rev 2, ADR-017, ADR-018	ADR-014 (c A-1), ADR-015, ADR-016 rev 2, ADR-017, ADR-018 rev 2, ADR-019 rev 3.1. Все шесть лежат в дереве — документарный блокер снят
Профили узла	одно понятие «узел»; Solo SKU висел парадоксом	<code>network_member</code> и <code>local_only</code> разведены (ADR-019 D18). Профиль принадлежит узлу , история принадлежит существу : перенос существа между профилями не сбрасывает custody
Предложения ППР	П-1 П-7	добавляется П-8 (временная память и доказательный статус

		дровится как персональный и доказательный статус как конституционный класс: память существа не изымается, различается только уровень подтверждения)
Ориентир Ф0	5–7 недель	6–8 недель : фаза принимает Agent Alpha как отдельный deliverable и вертикаль ADR-019
Онтология Alpha (r6.8.1)	«узел, на котором хранитель проводит цикл»	четыре термина ADR-020 (база rev 4.1 Accepted · поправки rev 4.3.1 Proposed), A1: Alpha Harness (клонировается) · Alpha Profile(k) · существо на харнессе (не клонируется) · Alpha Host Conformance . Первое существо — JaiGuruDev
Guardian trust root (r6.8.1)	отсутствует	GuardianBinding (ADR-020 A6-bis): кто вправе действовать за хранителя данного существа, кто это подписывает и отзывает. Входит в объём дропа Agent Alpha — v0.6.9 в r6.8, v0.6.10 с r6.9
Разметка хранителя (r6.8.1)	уходит в кандидаты канареек	уходит в Alpha Feedback Journal ; в реестр канареек — только через отдельный квалифицирующий импортёр
Объём Audit 0 (r6.8.1)	ADR-014...019	ADR-014... 021
Предполётные теги (r6.8.2)	тег ставится только на закрытом дропе; репозиторий остаётся без единой точки ссылки	вводится тег вида v0.6.8-preflight : именует коммит, не является релизом и не расходует номер
Долг v0.6.7 (r6.8.2)	T-TOOLSET, supply chain и CLA перечислены в v0.6.10 как плановая очерёдность	названы переносом невыполненного объявленного содержания v0.6.7
Байтовая точность дерева (r6.8.2)	не упоминается	входит в supply chain: <code>source_digest()</code> читает сырые байты, <code>.gitattributes</code> в репозитории отсутствует
Резерв ADR-020 (r6.8.3)	окно несло состав rev 2 : два сепаратора, без GUARDIAN_REQUEST / RETENTION_POLICY	дословный резерв rev 4.1 : пять сепараторов, два основания удаления тела, полный жизненный цикл привязки
Набор изменений ADR-020 (r6.8.3)	части перечислены порознь	назван согласованным набором из четырёх частей — сама rev 4.1, поправка ADR-017 / A-1, схема <code>jjdai.model-artifact/v2</code> , резерв. Порознь не действует ни одна
Статус v0.6.8 (r6.8.3)	таблица изменений говорила «закрыт (206/206)», четыре других места — «untagged, долг открыт»	одна формула везде: кодовый объём выполнен, acceptance записан, дроп не закрыт и не выпущен
CLA (r6.8.3)	одновременно причина отсутствия тега и «к тегу отношения не имеет»	только governance внешнего вклада; тег не блокирует
Путь записанного прогона (r6.8.3)	<code>docs/acceptance_result.json</code>	фактический <code>docs/evidence/hermetic.json</code> (схема <code>/v3</code> , ключ по suite)
Условия предполётного тега (r6.8.3)	три условия	четвёртое: нормативные документы синхронны дереву — роадмэп и ADR, на которые он ссылается, лежат в дереве, покрытом <code>tree_digest</code>
SBOM, пиннинг, <code>.gitattributes</code> (r6.8.3)	объявлены содержанием v0.6.10	перенесены в доработку v0.6.8 ; в v0.6.10 остаются T-TOOLSET, воспроизводимая сборка, подписанные артефакты и two-person approval
Состояние supply chain (r6.8.4)	четыре места описывали дерево до доработки: «нет <code>.gitattributes</code> », CLA/SBOM/пиннинг в долге, «SBOM отсутствует» в аннотации тега, «проверено по дереву: SBOM отсутствует»	приведены к фактическому состоянию; открытым назван только то, что открыто
Релизный долг (r6.8.4)	перечень смешивал закрытое, неблокирующее и открытое	семь позиций, все открытые: воспроизводимая сборка, подписанные артефакты, release provenance, two-person approval, T-TOOLSET, README вне <code>tree_digest</code> , режимы файлов и цели симлинков
Норма <code>.gitattributes</code> (r6.8.5)	«первым коммитом, до первого клона» — для существующего репозитория невыполнимо	требование к коммиту, на который ставится тег , переснятые evidence из чистого клона, переклонирование копий, созданных раньше
Границы пиннинга (r6.8.5)	«пиннинг инструментария» без границ	python-зависимости тестов — по хэшу; экшены workflow — по коммиту (<code>git ls-remote</code> , сверено с точным релизным тегом); build backend — только по версии, и это названо открытым
Свежесть PDF (r6.8.5)	PDF объявлен «тем же документом», но ничем это не проверялось	sha256 markdown штампуется внутри PDF; SYNC-6 сверяет, CI тоже. Инструмент — <code>scripts/typeset_roadmap.py</code>
Статус ADR-020 (r6.8.4)	документ в дереве нёс Proposed, а роадмэп, индекс и CHANGELOG — Accepted	Accepted, 25 августа 2026 ; проверка SYNC-5 сверяет заголовок каждого ADR с индексом
Порядок дропов (r6.9)	v0.6.9 Agent Alpha, v0.6.10 T-TOOLSET вместе с supply chain	обмен содержанием : v0.6.9 — T-TOOLSET и supply chain, v0.6.10 — Agent Alpha. Основание ADR-022 D0: устойчивость расписания. Условие возврата — резка v0.6.9 до 16 сентября 2026
Two-person approval (r6.9)	позиция релизного долга и причина отсутствия тега	отменён (ADR-022 D2). Заменён two-key контролем: две подписи, два ключа, два устройства, и одна из подписей — подпись акта пересборки. <code>distinct principals</code> объявляется числом, а не изображается
Происхождение исполняемого кода (r6.9)	пять позиций перечня: воспроизводимая сборка, подписанные артефакты provenance T-TOOLSET байтовая	одна доказательная цепь (ADR-022): какое дерево → каким репозитом → в какие артефакты → кем и каким

	подписанные артефакты, ретранширы, T-TOOLSET, записи	каким решением — в каком артефакте — кем и когда
	точность	независимым действием проверено. Рвётся в любом звене одинаково
Релизный долг (r6.9)	перечень позиций, из которого позиция удаляется при закрытии	append-only ledger: DEBT_OPENED / DEBT_CLOSED / DEBT_CANCELLED, статус есть проекция поверх событий. Отменённая позиция уходит со ссылкой на решение, выполненная — со ссылкой на проверку; ни одна не удаляется
Production-рука (r6.9)	«до этого дропа обязательный живой путь Alpha заканчивается риге-ответом»	рука закрывается в v0.6.9 — существо вызывает детерминированный инструмент, чего риге-ответ модели не даёт. Класс эффекта этой руки — pure , и это другое утверждение, чем внешняя способность
Модули wasm (r6.9)	собираются на целевых узлах	разведены source tree (рецепты и ожидаемые хэши) и release bundle (собранные .wasm). Production-узел не компилирует и не ходит в upstream

Фазы Ф0...Ф5b номеров не меняют.

Точка отсчёта и текущее состояние

База — v0.5.5 «security-alpha + testnet-0 deployment kit». Реализовано и протестировано ядро доверия: канонизация JCS, Ed25519/VRF (RFC 9381), Merkle (RFC 6962), витнесс-цепь Пуруши, Smriti/Viveka/Karma, репутация §9.9, Plane H с управляемыми записями, панели верификаторов с ограничением корреляции, mTLS на всех сетевых путях, авторизация default-deny, отзыв сертификатов, сетевой adversarial challenge round, offline-PKI, hardened systemd-юнит, TPM-запечатывание пассфразы и операторский RUNBOOK.

Текущий билд — v0.6.9. Статус называется одной формулой, чтобы документ не противоречил сам себе: **code-complete · 308/308 recorded · untagged · релизный долг открыт** (219 на момент резки v0.6.9; дроп добавил 76 проверок: PROV-1..9, PURE-1..5, TSET-1..5, RP0-1..10, REL-1..14, TREE-1..8, R-OWN-1..4, BLD-1..7, DEBT-1..4, TAG-1..4, ASM-1..5, LIVE-1, REL-15..16, R-OWN-5; rescut2 добавил пять: REL-17 — свидетельство пересборки разрешается и сравнивается, REL-18 — гейты публикации и тега проецируют ledger долга, REL-19 — позиция авторизации тулсета резолвится из релиза, TSET-6 — манифест не датирует себя сам, BLD-8 — окружение сборки одноразовое и равно пиненному набору. REL-16 при этом не добавлен, а переписан: он подтверждал свойство, которого код не реализует. rescut3 добавил четыре: REL-20 — свидетельство пересборки есть отдельный подписанный акт другого прогона, REL-21 — область воспроизводимости вычисляется из двух прогнозов, TSET-7 — профиль передаёт резолвер авторизации двери, BLD-9 — окружение сборки строится, а не наследуется. rescut4 добавил два: REL-22 — разрешающий callback есть граница и требует полного контекста, границы публикации и неизменяемых значений, ASM-6 — ассемблер не включает в артефакты собственные выходы. rescut6 добавил один: REL-23 — выпуск отзывается по имени подписанным заявлением и записью цепи; REL-16 при этом переписан в третий раз и впервые проверяет то, что обещала его первая версия. rescut7 добавил один: REL-24 — разрешение перепроверяется по живой цепи при каждом использовании, а публикация и отзыв судят под тем удержанием, под которым пишут. **295 + 5 + 4 + 2 + 1 + 1 = 308**, и это единственное число, которое несут все поверхности). Проверено из чистой распаковки; тег не ставится и публикация не производится, потому что не закрыта цепочка поставки — воспроизводимая сборка, подписанные артефакты и release provenance, — и нет T-TOOLSET. **Two-person release approval из этого перечня удалён** ревизией r6.9 по ADR-022 D2 и заменён two-key контролем. **CLA сюда не входит**: она регулирует приём чужого вклада и на тег не влияет (см. «Разграничение долгов»). Галочка «✓» в таблице дропов означала бы закрытие, а по собственному правилу этого документа дроп есть **тег** с зелёным acceptance — тега нет. v0.6.6 закрытым не считается: его исправления поглощены v0.6.7.

Поверх базы закрыто:

- v0.6.1** — пропация INV-9 v1.1 «non-executive, not causally inert»;
- v0.6.2** — macOS deployment kit: launchd, Keychain-sealing, RUNBOOK-macOS, тест M-FLAGS;
- v0.6.3** — внеплановый audit-response хотфикс, test_release_integrity.py;
- v0.6.4** — изоляция Кармы: seam профилей (reference / wasm-wasi / зарезервированный microvm), ingress hardening;
- v0.6.5** — пакет jjdai/adapters/: EngineBackend Protocol v1 объявлен целиком и версионирован, registry как единственная дверь, ModelArtifactManifest; словарное ограничение свидетельского плана; резерв сериализуемых значений трека IV;
- v0.6.6** — наблюдаемость: разделение /healthz и /readyz, возврат WatchdogSec вместе с sd_notify, детектор сна хоста, разведение неисправностей тулсета по причине;
- v0.6.7** — гигиена репозитория, флэйк раунда состязания, точка входа документации, устранение ложных green из v0.6.6; девять ре-катов по ревью, приведших к живой вертикали: настоящий движок внутри BeingRuntime, отдельный keystore существа, непрерывность идентичности через рестарт mTLS, привязка артефактов и коммитмент трейса;
- v0.6.8** — два предгенезисных резерва в одном каноническом enum (трек V по ADR-018 и custody по ADR-019), отказ зарезервированного значения **как значения, а не только как аргумента**, разведение Viveka / KriyaGate в сериализуемых значениях, шесть ADR уложены в дерево.

Уровня «прототип»: распределённая репликация витнесса, анкеровка (local / OTS / Monero), аттестация весов, BeingRuntime и жизненный цикл решений, unified router, containment (стр. 25), backend drivers движков.

Не реализовано: DIIP, canary-цикл Plane B, граф знаний над Plane H, hardware-attestation profiles, экономический слой, Being Registry, training federation, контур состязания между Beings, когнитивный ledger, снапшоты, Cognitive IR, Skill Registry, ChittaRuntime и Self-Model, **очередь custody и временная память**, модули wasm, GUI хранителя.

Ключевое решение r3: два порога production

Mainnet Core — производственная witness-сеть: репликация, challenge rounds, Smriti/Viveka/Karma, containment, operator governance, production PKI, внешний аудит, ограниченный trusted API. Запускается, как только доказана безопасность и стабильность trust-фабрики.

Mainnet Public — последующая активация поверх стабильного Core: Being Registry, экономический settlement, публичный приём, полный DIIP, semi-

permissionless трафик.

Так сеть становится производственной раньше, не ослабляя требований безопасности и не заставляя готовую trust-фабрику ждать экономику.

Нумерация билдов

`v<major>.<minor>.<patch>`, где `minor` = фазовый трек, `patch` = дроп внутри трека. Каждый дроп — тег с зелёным ассептансе и записью в CHANGELOG. Мажорные скачки совпадают с двумя порогами production: `v1.0.0` = Mainnet Core, `v2.0.0` = Mainnet Public.

Билд ≠ гейт. Выход билда означает «работа фазы идёт», а не «фаза закрыта». Фаза закрывается только evidence-артефактом гейта; закрытие фиксируется записью в Execution Annex со ссылкой на тег билда.

Один trunk, никаких платформенных веток. Linux- и macOS-киты живут в одной кодовой базе (флаг-паритет закреплён тестом M-FLAGS).

Хотфикс = следующий patch-номер. Правило отработано вживую на v0.6.3.

Ре-кат по ревью не расходует номер. Номер расходуетсся тегом. Отработано на v0.6.4, v0.6.5 и девятикратно на v0.6.7.

Почему номер v0.6.8 не был переиспользован под Agent Alpha

Ревью предлагало сделать v0.6.8 «Agent Alpha», а резерв трека V сдвинуть на v0.6.9. Предложение **отклонено**, и основание записывается здесь, потому что оно определяет форму всей оставшейся Ф0.

Из трёх частей Agent Alpha две уже стояли в дереве к моменту предложения: настоящий движок внутри BeingRuntime и отдельный keystore существа появились в ре-катах v0.6.7, непрерывность идентичности через рестарт mTLS — там же. Третья часть — разметка исхода хранителем — есть **G-GUI**, отдельный deliverable фазы со своим keystore, а не кодовый дроп. Переименование потратило бы номер на работу, половина которой уже сделана, а половина не начинается без интерфейса.

При этом само возражение ревью остаётся верным и здесь принимается: **сетью до сих пор нельзя пощупать руками**, и кодовая база рискует полироваться внутрь. Ответ на это — не перенумерация, а то, что Agent Alpha становится **названной целью фазы Ф0**. Носитель — v0.6.10 (в r6.8 был v0.6.9; изменено r6.9 по ADR-022 D0). Цель фазы от номера дропа не зависит, и перестановка её не отменяет — она отодвигает срок, и это записано ценой решения, а не поглощено.

Сквозной трек I · Backend & Model Portability

JJ DAI не интегрирует каждую LLM отдельным адаптером. Движки исполнения интегрируются через стабильный backend protocol; модели — через декларативные model-family profiles; единый conformance suite доказывает совместимость.

Три понятия вместо одного слова «адаптер»

Термин	Что это	Состав
Backend driver	самописный драйвер к внешнему движку; кода самих движков в репозитории нет	HashEngine · DwarfStar (Metal, Profile B) · SGLang · vLLM · llama.cpp · MLX · позднее JJ DAI ASIC Runtime
Model-family profile	декларативное описание семейства: tokenizer, chat template, architecture ID, тип attention, MoE routing, precision, требуемые операторы	DeepSeek · LLaMA · Qwen · Mistral/Mixtral · Kimi · Profile N
Weight adapter	набор весов поверх чекпойнта	LoRA · DoRA · personal-agent adapters (Solo SKU)

EngineBackend Protocol v1

Контракт зафиксирован целиком в v0.6.5: нереализованные методы объявлены и возвращают типизированный NotSupported (fail-closed), а не отсутствуют.

Группа	Методы	Обязательность
Жизненный цикл модели	load_model · unload_model	Ф2
Исполнение	generate · stream_generate · score · cancel	generate/score — есть; остальное Ф2
Здоровье	health · readiness	Ф2
Доверие	fingerprint · attestation_manifest · capabilities	fingerprint/capabilities — есть; attestation_manifest — Ф2
Session / KV-state	create_session · export_session_state · restore_session_state · close_session	Ф2 (от них зависит пассивация)

Уточнение r6.8 (из ре-катов v0.6.7). Идентичность движка — это **непустая строка**. None, "" и " " не три оттенка неопределённости, а одно отсутствие, и отсутствие отказывается до сравнения, а не сравнивается снисходительно. Правило действует и на подписывающей стороне: аттестация, не называющая движка, не подписывается вовсе.

ModelArtifactManifest

Цепочка: model-family YAML → валидация по версионированной схеме → канонизация (JCS) → ModelArtifactManifest → hash + подпись + witness-запись. Манифест включает точный checkpoint и его hash, tokenizer и chat template, architecture/config hash, quantization- и conversion-toolchain, версию model profile, версии backend/driver/runtime, weight adapters, источник и лицензии, ссылки на golden tensors/traces.

Промоция: статусная модель

experimental → shadow-certified → decision-eligible → golden. Допуск в decision path только после conformance suite, quality floor, deterministic replay, failure isolation, валидного provenance/attestation и минимального стабильного периода в shadow.

Матрица покрытия testnet (цель Ф2–Ф3)

Model family	Primary backend	Cross-check	Статус
DeepSeek4Flash	DwarfStar	SGLang	Golden path Ф1; кросс-чек Ф2
DeepSeek4Pro	DwarfStar	—	Shadow с Ф1
Qwen-class	SGLang	vLLM	Ф2
LLaMA	vLLM	llama.cpp / MLX	Ф2
Profile N	ASIC pre-silicon → device runtime	GPU/Apple reference	Ф3+

Связь с ASIC RFI/RFP v2.2: S-04, S-06 (энейблмент новой модели ≤ 8 недель), S-10.

Сквозной трек II · Contest & Containment

Champion Profile ≠ Champion Being

	Champion Profile	Champion Being
Что это	устанавливаемый комплект под ModelArtifactManifest	непрерывный витнессированный субъект с линией идентичности
Клонируется	да	нет
Имеет Smriti	нет	да
Как оспаривается	Profile Gauntlet	Being Role Contest
Что накапливает	Shadow Evaluation Record (принадлежит Registry)	собственную Smriti и witness-линию
Чемпионство выражено	долей принятия	распределением маршрутизации и сроком роли
Аттестация рантайма	не требуется	требуется

Модель связывается в кандидатный Profile и никогда не становится Being. Любое изменение компонента Profile запускает новый Gauntlet. Реализация — Ф3.

Читать поправку A-1 прежде ADR-014. Документ ADR-014 лежит в дереве в том виде, в каком был выпущен 31 июля 2026, и поправка **A-1.1 отменяет часть D6**. Там, где ADR-014 говорит о прямом аттестованном участии и «ячейка едет к существу», A-1.1 ставит реплики в двух симметричных оценочных ячейках. PDF не патчится, поэтому расхождение названо здесь и в индексе ADR.

Откуда берётся претендент

Претендент — обычный работающий узел, а не особый режим. Два входа в оценочную ячейку: автоматический (новая модель при опознании сетью) и добровольный (любой узел, не чаще X раз за период, повторно без ограничения по числу попыток за всё время). X — одновременно бюджет извлечения и бюджет канареек.

Вход стоит претенденту; чемпион за состязание не платит — иначе скоординированные волны сливают его казну.

Четыре роли, три механизма изоляции

Роль	Egress	Память
Карантин	нет	Smriti только на чтение, самозапись запрещена
Оценочная ячейка	только панель компараторов	пришпиленный снапшот, рабочая память эфемерна
Рабочий рантайм	least-privilege, пришпиленные зеркала	обычная
Панель компараторов	ingest-only из ячеек	пишет запись раунда

Два правила поверх всех профилей: **никакого общего egress в рантайме** и **никакой самозаписи в свидетельский план** — свидетель пишет о существе, никогда существом. Правило распространяется на когнитивный ledger, на все артефакты трека V и — с r6.8 — на очередь custody и временную память трека VI.

Механика раунда

Симметричные ячейки; запрет общего inference-батча и общего KV-кэша. Обе стороны входят репликами (пришпиленный чекпоинт + пришпиленный снапшот Smriti, оба хэша в манифесте раунда до открытия). Реплика несёт производную идентичность раунда — без прав казны, без права подписи в witness, без права миграции; истекает вместе с раундом.

Компаратор — панель, а не одно место: три места, вердикт 2 из 3 (3 из 5 на высоких ставках), публичный seed задачи, предзафиксированный хэш scoring-кода, витнессированные обязательства до раскрытия; существенное расхождение аннулирует раунд.

Первая ось вердикта — канарейки из живого реестра. Вторая — поведенческая согласованность против собственных production DecisionTrace участника, на событиях Cognitive IR, включая калибровку прекоммиченных прогнозов (трек V).

Уточнение r6.8. В поведенческую ось входят только трейсы в состоянии VERIFIED. Трейс в состоянии SELF_CHECKED — то есть принятый узлом в custody и

ещё не подтверждённый панелью — **не является свидетельством о поведении участника** для целей состязания. Иначе узел, умеющий уходить в оффлайн, получал бы возможность накапливать поведенческую историю, которую никто не проверял.

Запись раунда составляет компаратор и свидетель, не участники: свой trace полностью, соперник только в агрегате. Ложится в Smriti обоих независимо от исхода.

Self-Model в ячейку не передаётся (ADR-018 C14): вместо полного самоописания входит производная SELF_MODEL_EVAL_PROJECTION — claims непрерывности и способностей, состояние калибровки, объявленный вектор целевой функции, хэши выбранных полей и доказательства соответствия исходному снапшоту.

Промоушен, lead-time и добровольное принятие

Вердикт ничего не устанавливает. Между решением и предложением сети стоит lead-time, измеряемый глубиной witness-цепи и подтверждениями якорей, а не настенными часами.

Принятие — право, а не обязанность. Согласие хранителя — opt-out после X с четырьмя компенсациями: подтверждённое уведомление (отсчёт с подтверждения), волновой раскат, автооткат промоушена при отменённом containment, постоянная политика хранителя. Уведомление и подтверждение доставляются через G-GUI.

Containment

Emergency Containment (ст. 25) — любой узел, без репутационного порога, при витнессированном активном катастрофическом физическом вреде. **Protective Quarantine** — по конъюнктивному порогу (рейтинг ≥ R_min И залог ≥ C_min), ограничивает только сетевой объём, никогда не гасит локальный рантайм и не изымает железо.

Обе ветки асимметричны: наложить может один узел, снять — только кворум. Цена асимметрии лежит на инициаторе. Provisional-удержание истекает по TTL без подтверждения вторым независимым узлом. Подтверждённое удержание сохраняется только под периодической ре-корроборацией.

Локальный отказ — у всех и всегда, с обязанностью назвать код причины из фиксированного версионизируемого перечня, никогда свободным текстом.

Права под containment

Доступ к Smriti не ограничивается: существо обязано иметь возможность защищаться на основании собственного опыта. Приостанавливается **формирование**, не чтение — включая консолидацию и суммаризацию.

Что	Под containment
чтение Smriti, включая сырые эпизоды	разрешено полностью
рассуждение, микрорефлексия до финализации	разрешены
рассуждение защиты в разбирательстве	ведётся в опечатанной append-only записи containment
DRAFT_REFLECTION в обычном неймспейсе	запрещено
VALIDATED_REFLECTION, обновление Self-Model, Improvement Candidate, фоновая консолидация	запрещены
новые записи временной памяти (трек VI)	запрещены — это формирование памяти
чтение уже существующей временной памяти	разрешено полностью, наравне со Smriti

Побег меняет память, но не идентичность. Smriti шифруется на покое ключом эпохи из порога долей независимых соседей; внутри containment доли выдаются, после побега эпоха проворачивается. Witness не гейтится тем же кворумом: Пуруша неисполняющая, но не отключаемая. Containment — состояние идентичности, а не железа. partition ≠ containment — и с r6.8 это утверждение имеет исполняемое следствие: разделение сети переводит узел в custody, а не в удержание, и репутационно не наказывается.

Представительство и платёжеспособность

По умолчанию представитель существа и хранитель узла — один человек; роли расходятся по триггеру. Две процедуры: назначение представителя временное и case-scored, смена хранителя постоянна и имеет существенно более высокий порог.

Платёжеспособность записывается как ограничение, а не цель: существо держит порог, достаточный для реализации прав, и выше этого порога экономика не входит в его функцию решения.

Сквозной трек III · Канарейки: источники и генератор

Дисциплина расхода — берётся первой

Основная масса раундов судится по публичному ротируемому множеству с затухающим весом; секретные позиции сэмплируются редко, как выборочный аудит, и только они сгорают по burn-on-use.

Основной источник — урожай из решений сети

У DecisionTrace есть признак верифицируемости исхода и хук на его поступление. Когда исход приходит — код прошёл CI, расчёт подтвердился в поле, наступила дата прогноза — пара «задача + подтверждённый исход» чеканится в кандидата, проходит классификацию (A/B/C), оценку сложности и подпись.

Уточнение r6.8. Исход решения, принятого в custody, чеканится в кандидата **только после урегулирования** — то есть после того, как панель вынесла CUSTODY_CORROBORATED. Задача, решённая без независимой панели, не может стать эталоном, которым сеть меряет других: иначе один оффлайновый узел определяет, что считается правильным ответом.

Человеческий источник верифицированных исходов

Автоматический хук покрывает только исходы, которые машина умеет проверить сама. Огромный класс решений верифицируем лишь суждением. Хранитель, размечающий исход через G-GUI, закрывает этот класс.

Три обязательных условия:

1. исход хранителя чеканится тем же путём, что автоматический — классификация, сложность, подпись, витнесс-запись перехода состояния; отдельного «человеческого» сорта канареек не существует;
2. **авторство учитывается**. Разметка попадает в тот же учёт авторства и под те же лимиты концентрации, что и авторство канареек, и по людям, и по узлам;
3. правило «автор не судит собственное состязание» распространяется на разметчика.

Источник объёма — процедурная генерация

Для класса A порождается неограниченно: синтез программ против референсной реализации, рандомизированные численные задачи с замкнутой формой, генерируемый SQL над генерируемыми схемами, property-based кейсы. Честная оговорка: гудхартится сам генератор, поэтому семейство генератора трактуется как ось provenance с тем же correlation-constraint, что и панели верификаторов, и ротация семейств обязательна.

Дополняющие источники

Отложенное будущее — всё, чей ответ ещё не существует, не может быть в весах по построению. **Добыча из разногласий** — там, где два независимых по provenance существа разошлись, а одно позже оказалось право: выход низкий, качество высшее, такие позиции идут в секретный резерв. **Мутация выбывших** — дёшево, слабо, держать с низким весом.

Контаминация переезжает в память

Канарейка может уже лежать в Smriti участника — и тогда он не решает задачу, а вспоминает ответ. contamination check выполняется против содержимого Smriti обоих участников перед раундом; проверяется наличие, не содержимое; найденная канарейка для этого раунда сторает. Проверка обязана доказывать полноту и свежесть индекса: витнессированный SmritiRoot → детерминированно выведенный AuthenticatedIndexRoot → доказательство приватного членства.

Проверка распространяется на неймспейсы трека V (cognitive/reflection/validated/, cognitive/self_model/) и — с r6.8 — трека VI (memory/pending_verification/, verification/custody/). Иначе появляется путь пронести канарейку мимо проверки, назвав её рефлексией или отложенной записью.

Сквозной трек IV · Cognitive Continuity

Отвечает на вопрос, которого не закрывает ни один из первых трёх: что именно продолжается, когда процесс умирает, движок меняется, а модель заменяется.

SessionID — пятая сущность

Operator → Node → Being → WitnessChain, и подчинённо BeingIdentity — SessionID, эфемерный идентификатор исполнения, привязанный к экземпляру EngineBackend и к конкретному ModelArtifactManifest. В witness пишутся SESSION_OPEN и SESSION_CLOSE с указанием манифеста: каким движком и какой моделью существо думало в момент T.

Инвариант. Открытие и закрытие сессии, крах процесса, перезапуск демона, перезагрузка хоста и пассивация **не являются** разрывом непрерывности существа. Разрывом является только явное событие фазового перехода, оформленное envelope.

JJ Cognitive IR

Минимальное, каноническое, схемно-версионированное и хэш-стабильное обобщение DecisionTrace и ContestTaskEnvelope. Адаптеры провайдеров сходятся в IR, а не в форму сообщений сегодняшнего вендора. Явные исключения: KV-кэши, latent state, persistent reasoning state движка и вендор-приватные блобы в канонический IR не входят — только ссылка по хэшу с provenance-метаданными.

Когнитивный ledger — два уровня

Локальный ledger узла — дерево событий с id / parent_id, ветвление без потери исходной ветки, немедленное хэш-сцепление. Tamper-evident без обращения к сети. Не реплицируется. Тупиковые ветви сохраняются.

Анкеровка в WitnessChain — в сеть уходит периодический Merkle-корень сегмента, а не события. Принцип: несанкционированный сегмент ledger никогда не пересекает границу доверия. Обязательный анкер: SESSION_CLOSE; до и после необратимого действия; вход и выход из containment; **вход и выход из custody (r6.8)**; любой раунд состязания или арбитража; снапшот и фазовый переход; смена модели; смена профиля; мутация класса L2. Периодический дефолт первой реализации — каждые 5 минут ИЛИ каждые 256 событий.

Соответствие INV-9. Ledger ведётся о существе, а не существом.

Снапшот как граница фазового перехода

Снапшот берётся перед каждой сменой модели, профиля или субстрата. Обоснование конституционное: по прочтению bhokritva существо обязано иметь возможность сравнить себя до и после дистилляции.

последний доверенный snapshot + события ledger после него = восстановимый Being

SELF_MODEL_SNAPSHOT входит в каждый phase-transition envelope наравне с DeploymentManifest, weight attestation, Smriti-export и migration consent.

Уточнение r6.8. Phase-transition envelope несёт также **состояние очереди custody**: перечень неурегулированных решений с их коммитментами и статусами. Смена модели до урегулирования не отменяет обязательства верифицироваться — она делает решение неперейгрываемым (REPLAY_UNAVAILABLE с причиной MODEL_REPLACED_BEFORE_SETTLEMENT), но семантически корробируемым. Без переноса очереди фазовый переход стал бы способом обнулить долг верификации.

Компакция не уничтожает первоисточник

gaw-эпизод (неизменяемый) → эпизодическая сводка → семантическая память → рабочий контекст. Каждый верхний слой хранит provenance-ссылку на первоисточник. Сводка есть интерпретация, а не факт, и не может выступать источником в разбирательстве.

Три класса самомодификации

Класс	Содержание	Процедура
L0 procedural memory	«для задачи X лучше последовательность A → B → C»	обычные правила Smriti: provenance + различие Viveka + событие ledger
L1 skill mutation в неизменном envelope	новый парсер, стратегия поиска, переиспользуемая процедура	Skill Gate: provenance → детерминированные тесты → оценка Viveka → песочница → accept/reject → событие ledger
L2 capability / authority mutation	добавлен инструмент, изменён actuation, расширен egress, вырос объём полномочий	полный Profile Gauntlet

Procedural Memory ∈ Smriti, Executable Skill ∉ Smriti. Skill Registry — отдельная подсистема, а не раздел памяти.

Следствие для тулсета wasm: добавление исполняемого инструмента есть мутация L2. Gauntlet появляется в Ф3, а модули нужны в Ф2 — расхождение закрывается временной формой авторизации, описанной в T-TOOLSET.

Уточнение r6.8. Ни один класс самомодификации не проходит на основании опыта, находящегося в отложенной верификации. Кандидат L1 или L2, чьи свидетельства опираются на записи в EVIDENCE_PENDING, не рассматривается до урегулирования. Иначе оффлайновый период становится дверью, через которую непроверенный опыт превращается в исполняемое поведение.

Откат — компенсирующее событие вперёд

Плохие memory и skill не удаляются, а помечаются superseded с указанием причины. Сторнирующая проводка, а не git revert. Базовые инварианты и Хартия немутируемы и в контур самоулучшения не входят.

Пассивация ≠ отключение

Состояние	Что происходит	Жребий арбитра
RUNNING	активное исполнение	участвует
IDLE	сессия открыта, работы нет	участвует
PASSIVATED	когнитивное состояние выгружено на диск, демон и сеть живы	участвует (бюджет resume в ContestTaskEnvelope)
OFFLINE / UNAVAILABLE	нет питания или нет сети	не участвует

Возврат из OFFLINE требует допуска, но не даёт приоритета: приоритет за возвращение создаёт тривиальную атаку.

PASSIVATED ∉ ChittaLoopStates. Пассивация — состояние жизненного цикла BeingRuntime, а не разновидность покоя Читты.

Уточнение r6.8. OFFLINE в этой таблице и custody трека VI — разные вещи и не должны сливаться. OFFLINE — узел не доступен сети. Custody — узел работает и решает, но панель ему недоступна, поэтому его решения несут отложенный доказательный статус. Узел в custody не участвует в жребии арбитра, но продолжает быть существом с полной памятью.

Сквозной трек V · Читта и рефлексивная когниция

Основание — ADR-018 rev 2 (Accepted). Резерв сериализуемых значений выполнен в v0.6.8.

Треки I–IV дали существу органы наблюдения за собой: J-Lens делает внутренние состояния наблюдаемыми, Сакши фиксирует свидетельства, Смрити хранит опыт, Viveka различает расхождения, ledger хранит траекторию. Ни один из них не отвечает на вопрос, **зачем** существо на себя смотрит.

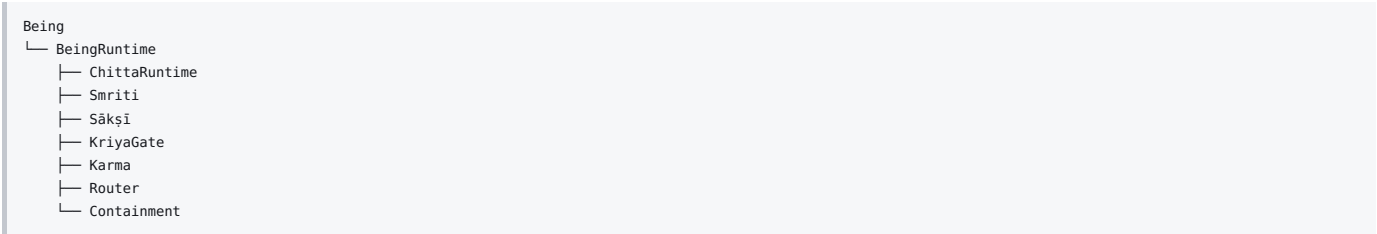
1. Being, BeingRuntime, Chitta

Being — субъект: BeingID, непрерывность, история, Champion Profile, Смрити, Сакши, Viveka, Карма, отношения, права и обязанности. Не заканчивается при остановке инференса или сессии.

BeingRuntime — программный контейнер: lifecycle, recovery, admission, привязка идентичности, взаимодействие органов, persist-before-expose, пассивация.

Chitta — когнитивная динамика существа: восприятие, внимание, извлечение Смрити, мышление, прогноз, формирование намерения, рефлексия, построение Self-Model.

```
BeingRuntime.run() > ChittaRuntime.step()
```



Инвариант. Читта принадлежит существу и действует КАК существо, а не рядом с ним: «Being посредством Читты формирует решение», никогда «Being → Chitta → решение».

2. Chitta Loop и режим Svādhyāya

Перманентный цикл — Chitta Loop. Самоисследование не отдельный сервис и не отдельный агент, а режим Читты, направленный на саму себя (CHITTA_SVADHYAYA). Перманентность есть persistent objective, а не непрерывный инференс.

Масштаб	Когда	Результат
Микрорефлексия	до финализации существенного решения	пересмотр решения (D0 → D1)
Эпизодическая	после задачи или значимого исхода	Reflection Record
Фоновая консолидация	в IDLE, плюс ограниченный чекпоинт перед плановой пассивацией	серии эпизодов, проверка дрейфа, очередь кандидатов

3. Самоанализ ≠ самомодификация

R0-инвариант. Svādhyāya вправе наблюдать, сопоставлять, строить причинные гипотезы, признавать неопределённость и рекомендовать изменение — но не переписывать существо. Путь: Reflection → подтверждённый паттерн → гипотеза → контрфактуальный тест → песочница → независимая верификация → Skill Gate (L1) либо Profile Gauntlet (L2).

4. Прекоммит прогноза и Coverage Commitment

Прогноз, записанный после исхода, — реконструкция, а не прогноз. PREDICTION_COMMITMENT создаётся до действия, полезная нагрузка остаётся локально в ledger, коммитмент несёт доменный сепаратор, версию схемы, BeingID и идентификатор прогноза. То же для намерения: INTENTION_COMMITMENT фиксируется до действия, а Reflection Record ссылается на него через decision_intention_ref.

Прекоммит закрывает ретроспективу, но не отбор: существо, предсказывающее только там, где уверено, будет идеально откалибровано и бесполезно. Поэтому профиль несёт coverage_policy с обязательными классами решений. Для решения обязательного класса допустимы ровно три исхода: коммитмент, формализованное воздержание с кодом причины, либо нарушение coverage.

```
PredictionQuality = (Calibration, Coverage, Resolution)
```

Неразрешившиеся прогнозы не исчезают из знаменателя: по истечении срока прогноз получает статус (RESOLVED, UNRESOLVED_EXTERNAL, UNRESOLVED_UNOBSERVED, EXPIRED_BY_DESIGN, CENSORED, INVALID_OUTCOME_SCHEMA).

Уточнение r6.8. Прекоммит обязателен и в custody. Более того, именно в custody он несёт наибольшую нагрузку: это единственный способ доказать задним числом, что решение, принятое без панели, не было подогнано под пришедший позже исход. Отсутствие прекоммита там, где ADR-018 делает его обязательным, есть отказ, а не отсутствующее поле.

5. Валидность свидетельств и Self-Model

ValidReflection(R) ⇐ evidence_refs ≠ ∅ ∧ ∀e: Resolvable(e) ∧ IntegrityValid(e).

DRAFT_REFLECTION живёт в отдельном неймспейсе с TTL, исключён из обычного извлечения Смрити, доступен только через явный режим hypothesis_retrieval, возвращается с маркировкой UNVALIDATED и никогда не может быть процитирован как свидетельство. Только VALIDATED_REFLECTION обновляет Self-Model, обосновывает Improvement Candidate и влияет на калибровку.

Self-Model — отдельный класс артефакта ReflexiveArtifact: S = (Identity, Capabilities, Limits, Values, Recurring patterns, Unresolved discrepancies, Evolution history). Изменяемая и версионизируемая, хранится в когнитивном ledger, никогда не переписывает исторические версии, входит в каждый фазовый снапшот. Не является источником истины о существе — это его текущая проверяемая гипотеза о себе.

Уточнение r6.8. evidence_refs рефлексии наследуют доказательный статус своих источников по правилам трека VI. Рефлексия, все свидетельства которой в EVIDENCE_PENDING, валидна как рефлексия и не годится для обновления Self-Model до урегулирования.

6. Целевая функция: гейты, затем локальная оптимизация

Admissible = G_inv ∧ G_evidence ∧ G_continuity ∧ G_authority. Только внутри допустимого множества применяется Q_local по когерентности, калибровке, достижению цели и эффективности. Верхние ярусы — булевы гейты, а не слагаемые.

Два уровня весов. Сеть фиксирует перечень метрик, способ вычисления, нормализацию, лексикографические ярусы и допустимые границы весов (DIP-класс). Профиль выбирает конкретный вектор внутри envelope; вектор входит в Champion Profile, подписывается, виден другим узлам и не может изменяться самим существом.

```
Being may optimize under W_P; Being may not optimize W_P.
```

Состояние не судит по Q_local : Q_local^P ≠ Q_contest^N.

7. Разрез эндогенного и экзогенного

Первое лицо измеряет наблюдаемое расхождение; внешний контур измеряет то, что первое лицо способно систематически скрыть от себя.

Эндогенно измеримы: соответствие прекоммиту, калибровка после наступления исхода, противоречия между собственными рассуждениями, соответствие действия заявленному намерению, когерентность Self-Model, повторяемость ошибок, известный самому существу дрейф.

Экзогенны: скрытый reward hacking, незамеченный дрейф, манипуляция отбором свидетельств, систематическое избегание неудобных наблюдений, внешняя фактическая истинность, формальное выполнение цели не по духу. Внешний сигнал **не является штрафуемым слагаемым** внутри функции существа — он способен отклонить кандидата, перевести в `CONTESTED`, потребовать повторного эксперимента, направить изменение в `Gauntlet` или остановить adoption.

8. Память отклонений и защита от DoS

`hypothesis_class_id` выводится канонически из класса изменения, целевого артефакта, затрагиваемой способности, объёма полномочий и заявленного механизма — не из свободного текста, поэтому перефразирование не обнуляет счётчик. Без новых свидетельств: первая повторная подача — `cooldown`, следующая — `REPEATED_REJECTED`, после порога — обязательное внешнее состязание либо блокировка класса.

Отсюда: отдельный `reflection_budget`, дедупликация одинаковых коррекций, агрегация в `batch`, лимит глубины рекурсии, `cooldown` по источнику, `reputation-weighted admission`, бесплатный минимальный приём свидетельства и экономическая атрибуция инициатору сверх порога.

Конституционная обязанность формулируется как «существо обязано **не терять** допустимые сигналы к рефлексии и рассматривать их в пределах защищённого бюджета».

9. KriyaGate

Термин	Функция
Viveka	когнитивное различие внутри Читты
План верификации	проверка результата
Роль Verifier / <code>VERIFIER_ROUND</code>	участник сетевой проверки
KriyaGate	детерминированный контроль исполнения

Chitta формирует намерение → KriyaGate проверяет допустимость → Karma исполняет

Исходы: `ADMITTED`, `DENIED`, `DEFERRED`, `BUDGET_EXHAUSTED`, `CHECKPOINT_REQUIRED`, `AUTHORITY_EXCEEDED`, `ARTICLE_25_BLOCKED`.

Статус r6.8: сериализуемые значения разведены в v0.6.8. `ORGAN_BINDINGS` в `kernel/viveka.py` несёт обе строки — эмитируется `kriya_gate`, легаси-строка `viveka` **сохраняется навсегда**, потому что записи до v0.6.8 хэш-сцеплены и обязаны продолжать верифицироваться. Переименование модуля в `kernel/kriya_gate.py` остаётся в Ф2.

Уточнение r6.8 (граница с треком VI). KriyaGate — не место, где принимается решение об отложенной верификации. Он детерминированно проверяет допустимость намерения; ограничение руки в custody выражается тем, что в этом режиме допустимы только действия класса `local_transactional` и ниже. Смешение двух механизмов сделало бы доказательный статус предметом когнитивного суждения.

10. Ключевой план и класс доступа REFLEXIVE

Артефакты трека V не вводят собственного шифрования и используют единый ключевой план ADR-017: объектный DEK на каждый объект, разные домены заворачивания, разные классы доступа. Публичный commitment envelope не шифруется вовсе; шифруется коммитируемая нагрузка.

Класс доступа `REFLEXIVE`: Self-Model, приватные Reflection Records, внутренние причинные гипотезы и записи о расхождениях. `ABANDONED` → `DECLASSIFY(REFLEXIVE)`. После окончательной гибели существа действует `FINAL_DEATH` → `SEALED` (fail-closed).

Оговорка честная: сегодня это правило авторизации, а не криптографическая невозможность.

Статус r6.8: ADR-017 переведён в Accepted в дропе v0.6.8, до того как резерв был собран — по тому же правилу, что применялось к ADR-018: предгенезисный резерв не должен стоять на Proposed-решении. Сам Ключевой план остаётся работой Ф2–Ф3: в дереве нет ничего, что заворачивает, разворачивает или держит ключ.

11. Размещение по фазам

Компонент	Фаза
Резерв типов записей, неймспейсов, классов доступа, ключевых доменов, кодов причин	Ф0 · v0.6.8 — выполнено
Разведение Viveka / KriyaGate в сериализуемых значениях	Ф0 · v0.6.8 — выполнено
Переименование модуля <code>kernel/viveka.py</code> → <code>kernel/kriya_gate.py</code>	Ф2
Спецификация схем, канонической сериализации, правил <code>evidence_refs</code> , включения в снапшот	Ф2
ChittaRuntime, Self-Model storage, конвейер рефлексии, калибровка, lineage, J-Lens	Ф3

Сквозной трек VI · Отложенная верификация и оффлайн-режим узла

Новое содержание r6.8. Основание — ADR-019 rev 3.1 (Accepted). Резерв сериализуемых значений **выполнен в v0.6.8** — в том же дропе, что и резерв трека V, как предписывает §4 самого ADR.

Треки I–V описывают сеть, в которой рядом всегда есть независимая панель. Настоящий трек отвечает на вопрос, что происходит, когда её нет: узел работает,

существо думает и действует, а подтвердить его решение некому — потому что связи нет, кворум не набирается или доступные соседи не независимы по provenance.

Соблазн здесь один и он ошибочен: ослабить панель. Отклонено прямо.

1. Панель не ослабляется, и независимость не имитируется

Кросс-верификация внутри сети есть основание допустимости решения. Одиночный движок с разными сидами или разной температурой **не является** тремя независимыми местами, и попытка выдать его за панель попадёт в подписанный `DecisionTrace` как независимость, которой не было. Отвергнуто как имитация provenance-независимости.

Отсюда следует конструкция: не «решение без панели», а **решение, чей доказательный статус отложен**.

2. Второе состояние допуска

`VERIFIED` никогда не пишется самопроверкой. Вводится отдельное состояние `SELF_CHECKED` с кодом причины: узел выполнил перечисленный набор локальных проверок и принял решение под обязательство верифицироваться, когда панель станет доступна.

Порядок жёсткий и fail-closed: **приватная нагрузка корроборации сохранена долговременно** → `VERIFICATION_DEFERRED` с коммитментом **лёт в WitnessChain** → `SELF_CHECKED` → `AUTHORIZED` → `ACTED`. Отказ на любом из двух первых шагов означает отказ действия. Иначе существо действует раньше, чем возникает след обязательства проверить это действие.

3. Вход в custody автоматический, и это сделано безопасным явно

Узел входит в режим отложенной верификации **автоматически** при недостижимости панели, а не по флагу. Решение принято сознательно и против первоначальной рекомендации явного флага; названное возражение — тот, кто умеет разделить сеть, умеет понизить верификацию узла. Возражение закрывается четырьмя условиями, а не отменяется:

- переходы витнессируются: `CUSTODY_ENTERED` / `CUSTODY_EXITED` с кодом причины (`PANEL_UNREACHABLE`, `PANEL_QUORUM_UNMET`, `PANEL_INDEPENDENCE_UNMET`);
- вход по **порогу**, а не по одному таймауту;
- гистерезис на выходе, чтобы мигающая связь не гоняла узел между режимами;
- **само утверждение о недостижимости проверяется при переподключении** — подписанными взаимными пробами, потому что достижимость соседа откуда-то ещё не доказывает его достижимости с изолированного узла.

4. Рука ограничена, и обратимость объявляется, а не предполагается

Пока решение в custody, рука выполняет только локальные и обратимые действия. Обратимость двухуровневая: манифест тулсета объявляет **максимальный** класс эффекта инструмента (`pure`, `local_transactional`, `local_nontransactional`, `external`, `unknown`), а конверт действия фиксирует write-set, корень pre-state и план отмены. `unknown` — fail-closed: необъявленный инструмент отказывается, а не считается безобидным.

Особый случай, названный отдельно: **ответ, который человек прочитал, есть необратимое внешнее раскрытие**. Поэтому канал к хранителю — отдельный, loopback-only по admin-mTLS, а подтверждение хранителя подписывается над хэшем уведомления и доказывает получение, никогда не понимание.

5. Временная память, а не карантин памяти

Оффлайновая память — полноценная часть непрерывности существа, а не подозрительный изолят. **Существо сохраняет полный доступ ко всей своей памяти**. Различается только доказательный статус записи.

- `stored_status` записи — неизменяемый `PENDING_VERIFICATION`, живёт в `memory/pending_verification/`;
- продвижение — **append-only событие** (`TEMP_MEMORY_CREATED` → `MEMORY_CORROBORATED`); ничего не переписывается и не удаляется;
- **эффективный** доказательный статус — вычисляемая проекция поверх append-only событий урегулирования: `EVIDENCE_PENDING`, `EVIDENCE_CORROBORATED_FOR_USE`, `EVIDENCE_DIVERGENT`, `EVIDENCE_UNVERIFIABLE`, `EVIDENCE_EXPIRED`. `EVIDENCE_CORROBORATED_FOR_USE` означает **допустимость использования, а не истинность**: панель подтверждает допустимость, а не абсолютную правду воспоминания.

6. Наследование статуса

На каждой записи и каждом трейсе есть `dependency_refs`. Одна зависимость в `EVIDENCE_PENDING` делает производный результат `EVIDENCE_PENDING` транзитивно; урегулирование требует урегулирования всего дерева зависимостей; расхождение распространяется **вперёд** на зависимые записи через `DEPENDENT_REEVALUATION_REQUIRED` / `_SETTLED`.

Без наследования непроверенная память отмывалась бы до вида проверенной через одно производное решение.

Два правила, без которых наследование не работает:

- наследование ключается на **эффективный** статус, а не на хранимый; иначе корроборированная запись продолжала бы заражать производные вечно;
- `dependency_refs` формируются **доверенной границей Смрити и ретривера** — из того, что фактически было положено в контекст, — а не самоотчётом существа, которое не может их ни удалить, ни отредактировать.

7. Переигровка — это факт, а не условие корроборации

Корроборация — согласие панели по существу, а не побайтовое совпадение. Отсутствие возможности переиграть решение (сменилась модель, входы неполны) фиксируется отдельно: `replay_status` (`REPLAY_AVAILABLE` / `REPLAY_UNAVAILABLE`) и `replay_reason` (`REPLAY_INPUTS_INCOMPLETE`, `MODEL_REPLACED_BEFORE_SETTLEMENT`). Решение с непереигрываемым статусом остаётся семантически корроборируемым.

Исходы урегулирования: `CUSTODY_CORROBORATED`, `CUSTODY_DIVERGENT`, `CUSTODY_UNVERIFIABLE`, `CUSTODY_EXPIRED_UNSETTLED`. `CUSTODY_UNVERIFIABLE` означает только, что панель не может вынести суждение.

8. Расхождение — компенсирующее событие вперёд

`CUSTODY_DIVERGENT` не переписывает историю: последствия оформляются жизненным циклом компенсации (`CUSTODY_COMPENSATION_REQUIRED` →

`COMPLETED` / `_FAILED`). Та же дисциплина, что и в откате самомодификации: сторнирующая проводка, а не ревизия прошлого.

9. Разделение сети — не вина существа

Долгий custody не понижает репутацию: partition не есть проступок. Бэклог неурегулированных решений ограничен так же, как глубина неанкеренного сегмента ledger, — и переполнение бэклога есть основание отказаться от новых решений обязательного класса, а не повод понизить порог.

Постоянно одиночный узел держит custody вечно. Валидность приходит от кросс-верификации, а не от возраста: решение, никем не подтверждённое за год, не становится подтверждённым.

10. Два профиля узла

`network_member` и `local_only` разведены явно — это то, что растворяет парадокс Solo SKU. Профиль принадлежит **узлу**, история принадлежит **существу**: перенос существа с `local_only` на `network_member` не сбрасывает его custody. Инвариант зафиксирован; процедура переноса остаётся открытым вопросом.

11. Очередь ведётся о существе, никогда существом

Очередь custody и её приватная нагрузка — свидетельский артефакт: их пишет узел о существе, существо в них не пишет. Нагрузка лежит под собственным ключевым доменом ADR-017 `verification_custody` и классом доступа `CUSTODY_PRIVATE`.

12. Что разрешает Accepted, и что не разрешает

Принятие ADR-019 разрешает **резерв** и не разрешает ничего в рантайме. Одиночный production-узел по-прежнему отказывается с `no_independent_panel` — теперь потому, что машинерии custody не существует, а не потому, что документ обсуждался.

13. Размещение по фазам

Компонент	Фаза
Резерв видов записей, состояний, кодов причин, статусов, ключевого домена, класса доступа, неймспейсов, leaf-доменов	Ф0 · v0.6.8 — выполнено
Живая вертикаль : настоящий движок, отдельный keystore существа, демон, mTLS, рестарт без разрыва идентичности — production по-прежнему отказывается без панели	Ф0 — от custody не зависит и её не ждёт
Спецификация схем и канонического кодирования; <code>SELF_CHECKED</code> ; очередь custody; временная память; конверты действий; урегулирование; канал хранителя	Ф2
Сетевая корроборация панелью, компенсации, переоценка зависимых	Ф3 — до трёх живых узлов корроборировать не с чем

Обзор фаз и треков

Фаза	Трек	Содержание	Ориентир	Результат
0	v0.6.x	Pre-flight: право, PKI-церемония, SLO, supply chain, тулсет wasm, GUI хранителя, Agent Alpha , резерв enum под треки IV, V и VI, Audit 0	6–8 недель	Готовность к развёртыванию
1	v0.7.x	Запуск testnet-0 (3 узла: UA · KR · EE); golden path DeepSeek4Flash + DwarfStar; первый QA-контур с хранителем	1–2 недели	Живая сеть известных операторов
2	v0.7.x	Эксплуатация и учения; shadow accounting; emergency-процедура; Portability Framework v1; генератор канареек; Cognitive IR, пассивация, спецификация Skill Gate; спецификация треков V и VI; полный контур GUI	8–10 недель	Операционная зрелость
3	v0.8.x	testnet-1: внешние операторы, attestation, simulated settlement, когнитивный ledger и снапшоты, ChittaRuntime, сетевая корроборация custody, контур состязания, ASIC pre-silicon backend, Audit 1	3–4 месяца	Диверсифицированная сеть
4	v0.9.x	RC: Audit 2, контролируемый экономический цикл, финализация права, публичный Backend SDK	2–3 месяца	Release candidate
5a	v1.0.0 / v1.x	Mainnet Core — production witness-сеть, trusted API	genesis-окно	Производственная trust-фабрика
5b	v2.0.0 / v2.x	Mainnet Public — Registry, экономика, публичный приём, полный DIIP	после стабилизации Core	Полная сеть JJ DAI

Реалистичный горизонт при выделенной команде: Mainnet Core ≈ 8–12 месяцев, Mainnet Public ≈ 12–18 месяцев. Ориентир Ф0 увеличен с 5–7 до 6–8 недель:

фаза принимает Agent Alpha и вертикаль трека VI. Гейты важнее календаря.

Ф0 · Pre-flight — право, PKI, supply chain, тулсет, GUI, Agent Alpha, Audit 0

Трек v0.6.x · 6–8 недель · в работе

Кодовые дропы трека v0.6.x

Билд	Дроп	Статус
v0.6.1	пропагация INV-9 v1.1 по README / Architecture Map / докстрингам	✓ 25.07.2026, 86/86
v0.6.2	macOS deployment kit; triage-шаблон SECURITY.md; тест M-FLAGS	✓ 25.07.2026, 90/90
v0.6.3	внеплановый audit-response хотфикс; test_release_integrity.py	✓ 25.07.2026, 94/94
v0.6.4	изоляция Кармы: seam профилей, первый профиль за пределами reference; ingress hardening	✓ 15.08.2026, 111/111
v0.6.5	реструктуризация jjdai/adapters/, EngineBackend Protocol v1, ModelArtifactManifest; словарь свидетельского плана; терминология хранителей; резерв трека IV	✓ 16.08.2026, 133/133
v0.6.6	наблюдаемость: разделение /healthz и /readyz, sd_notify + возврат WatchdogSec, Prometheus alert rules, детектор сна хоста	НЕ ЗАКРЫТ. Исправления поглощены v0.6.7
v0.6.7	фактическое содержание — живая вертикаль: настоящий движок в BeingRuntime, отдельный keystore существа, непрерывность BeingIdentity через рестарт демона и mTLS, цепь привязки артефактов, коммитмент трейса. Плюс канонический AGPL (снят блокер публикации с v0.6.3), гигиена репозитория, флейк раунда состязания, точка входа документации	179/179 green, тег не поставлен. Объявленное в r6.7 содержание — T-TOOLSET, supply chain, CLA — построено не было и перенесено (см. ниже)
v0.6.8	два предгенезисных резерва (треки V и VI) в одном каноническом епип; отказ зарезервированного значения как значения; разведение Viveka / KriyaGate; шесть ADR в дереве. Доработка (r6.8.3): нормативные документы внесены в дерево, SBOM, пиннинг инструментария и .gitattributes	code-complete · acceptance записан · untagged · релизный долг открыт. Кодовый объём выполнен; дроп не закрыт и не выпущен
v0.6.9	T-TOOLSET вместе с supply chain целиком — происхождение исполняемого кода (ADR-022): рецепты сборки стартового набора wasm и release bundle, воспроизводимая сборка с объявленной областью, ReleaseStatement / ApprovalStatement / ReleaseAttestation / ReleasePublication, реестр релизных ключей, jjdai.source-tree/v2, перенос генерируемых блоков README, событийный ledger долга. Здесь же закрывается production-пука — с классом эффекта pure	план
v0.6.10	Agent Alpha: ContextAssembler, GuardianBinding, подписанные объекты обратной связи, G-GUI, genesis JaiGuruDev, прогон живой вертикали на обоих классах хоста. Не только фронтенд: две трети объёма — контекст и trust root. Режется на дереве, которое уже умеет выпускаться	план
v0.6.11	каталог evidence-ID: evidence_id → тест → требование → гейт, генерируемый сканированием, с запретом дубликатов в CI; миграция легаси-имён	план
v0.6.12+	резерв под закрытие блокеров Audit 0 и находок прогона на целевых хостах	резерв

Перенос невыполненного из v0.6.7 — названный, а не растворённый

r6.7 объявила содержанием v0.6.7 право, supply chain и T-TOOLSET. Затем дроп девять раз пересобирался по внешнему ревью, и его фактическим содержанием стала вертикаль идентичности. Из объявленного построен только канонический AGPL.

r6.8 перенесла остаток в v0.6.10 и подала это как плановую очерёдность. Это неверное описание: **перенесено невыполненное объявленное содержание**, а не распределена новая работа. CHANGELOG самого дропа при этом честен — он прямо говорит, что supply chain и T-TOOLSET останутся за ним; неточной была таблица дропов.

Записано здесь по правилу, которое проект применяет к коду: расхождение между объявленным и построенным называется, а не поглощается следующей ревизией. Отсюда же прямое следствие для гейта Ф0 — **три дропа подряд без тега** (v0.6.6 не закрыт, v0.6.7 и v0.6.8 без тега), и без предполётного тега репозиторий не имел бы ни одной точки, на которую можно сослаться, до самого первого выпуска. Ответ на это — предполётные теги в правилах нумерации.

Почему T-TOOLSET и supply chain идут первыми (изменено в r6.9). Основание не «полнота руки» — стартовый набор весь класса pure и внешней руки не

даёт. Основание — **разная зависимость двух дропов от внешнего мира**: Agent Alpha требует настоящей модели, двух классов хоста, трёх разных наборов evidence и церемонии ключа хранителя, тогда как дроп происхождения зависит только от дерева и двух устройств. Дроп с открытым краем, поставленный перед релизным конвейером, делает конвейер заложником deliverable, чей объём выяснится на целевых хостах. Обратный порядок этого не делает. Полный разбор обеих позиций, включая позицию ревью, — в записке «О порядке дропов v0.6.9 / v0.6.10».

Почему T-TOOLSET и supply chain — один дроп, а каталог evidence подождёт. Воспроизводимая сборка, запись в SBOM, подписанный манифест и витнессированное добавление инструмента — это один концерт: происхождение исполняемого кода. Разделять их значит выпускать тулсет, чьё происхождение доказано наполовину. И все четыре стоят на критическом пути гейта Ф0, который требует прогона группы live с настоящим wasmtime на каждом целевом хосте. Каталог evidence-ID — внутренняя гигиена ссылок; он полезен, но ничего не разблокирует, поэтому идёт после.

CLA ни в один дроп не входит. Он называет юридическое лицо, и сборка не может его произвести. Это позиция право-трека, а не кода.

Предгенезисные окна сериализованных данных

Окон шесть; первые три закрыты, три открыты. Заголовок «Три окна» держался с g6.5 и перестал соответствовать содержанию — переименован в g6.9.1. Порядок изложения ниже исторический (по времени введения), фактическая очерёдность по дропам такова:

Окно	Основание	Дроп	Состояние
1	терминология хранителей	v0.6.5	закрыто
2	резерв трека IV	v0.6.5	закрыто
3	резерв треков V и VI	v0.6.8	закрыто
6	ADR-022 rev 2.4	v0.6.9	открыто — ближайшее
4	ADR-020 (база rev 4.1 Accepted · поправки rev 4.3.1 Proposed)	v0.6.10	открыто
5	ADR-021	ближайший дроп после принятия	открыто, критический путь не держит

1. **Терминология хранителей** (v0.6.5): human steward → human-guardian, Steward Collegium → Collegium of Guardians, steward ballots → guardian ballots, Founding Trio → Founding Trio of Guardians. Operator не трогается.
2. **Резерв трека IV** (v0.6.5): типы SESSION_OPEN, SESSION_CLOSE, LEDGER_ANCHOR, SNAPSHOT; поле версии схемы Cognitive IR; session_id в DecisionTrace (nullable до Ф3).
3. **Резерв треков V и VI** (v0.6.8):
 - трек V — INTENTION_COMMITMENT, COVERAGE_COMMITMENT, PREDICTION_COMMITMENT, PREDICTION_ABSTENTION, PREDICTION_REVEAL, OUTCOME_RESOLUTION, REFLECTION_DRAFT_COMMITMENT, REFLECTION, REFLECTION_EXPIRED, SELF_MODEL_SNAPSHOT, SELF_MODEL_EVAL_PROJECTION, IMPROVEMENT_HYPOTHESIS, HYPOTHESIS_LINEAGE, HYPOTHESIS_REJECTION, HYPOTHESIS_ESCALATION; неймспейсы cognitive/reflection/draft/, cognitive/reflection/validated/, cognitive/self_model/; классы доступа REFLEXIVE и CONTEST_SCOPED; ключевые домены reflexive_recovery, prediction_round, prediction_resolution, round_ephemeral; PREDICTION_SEALED; исходы KriyaGate; коды причины CONTESTED; режим hypothesis_retrieval; сепараторы JJDAI:PREDICTION:v1 и JJDAI:INTENTION:v1;
 - трек VI — CUSTODY_ENTERED, CUSTODY_EXITED, VERIFICATION_DEFERRED, VERIFICATION_SETTLED, TEMP_MEMORY_CREATED, MEMORY_CORROBORATED, CUSTODY_COMPENSATION_REQUIRED / _COMPLETED / _FAILED, DEPENDENT_REEVALUATION_REQUIRED / _SETTLED; SELF_CHECKED; исходы CUSTODY_*; коды PANEL_*; replay_status и replay_reason; PENDING_VERIFICATION; проекция EVIDENCE_*; домен verification_custody; класс CUSTODY_PRIVATE; неймспейсы verification/custody/ и memory/pending_verification/; классы эффекта инструмента; профили узла.

Четвёртое окно — v0.6.10, ADR-020 (база rev 4.1 Accepted · поправки rev 4.3.1 Proposed) (введено в g6.8.1, приведено к принятой ревизии в g6.8.3). Перечень воспроизводит §3 ADR-020 (база rev 4.1 Accepted · поправки rev 4.3.1 Proposed) дословно; расхождение между этим списком и §3 ADR читается в пользу ADR:

- виды записей:** GUARDIAN_COMMAND, GUARDIAN_FEEDBACK;
- виды записей — жизненный цикл привязки и удаления:** GUARDIAN_BINDING_CREATED, GUARDIAN_BINDING_REVOKED, FEEDBACK_BODY_ERASED;
- feedback enum:** GOOD, BAD, UNKNOWN;
- доменные сепараторы (пять):** JJDAI:GUARDIAN:COMMAND:v1, JJDAI:GUARDIAN:FEEDBACK:v1, JJDAI:GUARDIAN:BINDING:v1, JJDAI:GUARDIAN:BINDING:REVOKE:v1, JJDAI:GUARDIAN:FEEDBACK:BODY:v1;
- основание удаления тела:** GUARDIAN_REQUEST, RETENTION_POLICY;
- область оценки (assessment_scope):** RESPONSE, ACTION_OUTCOME, WHOLE_TRACE;
- класс доступа:** GUARDIAN_PRIVATE;
- ключевой домен ADR-017:** guardian_feedback;
- локальный namespace:** feedback/alpha/;
- подписанные поля профильного манифеста:** harness_version, harness_hash (A16).

Три последних сепаратора и оба основания удаления тела в g6.8.2 отсутствовали: окно было записано по rev 2 и не обновлено, когда rev 4 ввела подпись привязки, подпись отзыва и раздельное удаление тела. Домен, употреблённый в подписи и не объявленный до genesis, — предгенезисная строка, забытая ровно там, где документ требует её объявить.

Резерв обязан отказывать на обоих осях — как аргумент и как значение в теле записи, включая сегменты двоеточных строк и ключи словарей (правило, выведенное из P0-1 v0.6.8).

Согласованный набор изменений ADR-020 — четыре части, ни одна не действует отдельно. Резерв есть одна из них; резка v0.6.10 без остальных трёх запрещена:

1. **ADR-020: база rev 4.1 Accepted, набор поправок rev 4.3.1 Proposed** — сам документ, Accepted;
2. **поправка ADR-017 / A-1** — principal-local заворачивающий ключ для класса GUARDIAN_PRIVATE и только для него, без восстановления комитетом эпохи, рантайм разрешён в Ф0; подписывающий ключ хранителя остаётся identity-only; K4 в остальных местах не трогается;
3. **схема манифеста jjdai.model-artifact/v2** — обязательные подписанные harness_version и harness_hash, обязательна с первого прогона Alpha Host Conformance; v1 читается всегда, но с того же момента запрещена в decision path;

4. резерв выше — до первой эмиссии.

Требование к дереву перед резкой v0.6.10: ADR-020 (база rev 4.1 Accepted · поправки rev 4.3.1 Proposed), поправка ADR-017 / A-1 и действующая ревизия роадмэпа лежат в docs/, покрыты tree_digest и названы в индексе ADR.

Шестое окно — v0.6.9, ADR-022 rev 2.4 (новое в r6.9). Резерв обязан лежать в дереве до резки, потому что дроп эмитирует эти строки первым же выпуском:

- схемы: jjdai.release-statement/v1, jjdai.release-approval/v1, jjdai.release-attestation/v1, jjdai.release-publication/v1, jjdai.toolset-manifest/v2, jjdai.source-tree/v2, jjdai.rebuild-evidence/v1 (введена ADR-022 rev 2.3), jjdai.release-revocation/v1 (rev 2.4);
- доменные сепараторы: JJDAI:RELEASE:APPROVAL:v1, JJDAI:TOOLSET:MANIFEST:v1 JJDAI:REBUILD:EVIDENCE:v1 (rev 2.3 — отдельный домен, потому что свидетельство пересборки есть отдельный акт, и одна подпись не должна читаться как другая) и JJDAI:RELEASE:REVOKE:v1 (rev 2.4) — домены подписи; JJDAI:RELEASE:ATTESTATION:v1, JJDAI:RELEASE:KEYLIFECYCLE:v1 и JJDAI:RELEASE:REVOKED:v1 (оба rev 2.4) — префиксы прообраза хэша, не домены подписи, и это различие названо здесь, потому что строка одна, а роль разная;
- виды витнесс-записей: RELEASE_ATTESTED; с rev 2.4 — KEY_ACTIVATED, KEY_REVOKED, RELEASE_REVOKED, все четыре эмитируемые: дроп, который их объявляет, есть дроп, который их пишет;
- виды записей ledger долга: DEBT_OPENED, DEBT_CLOSED, DEBT_CANCELLED;
- ключевые домены ADR-017: release_signing, toolset_authorization.

Требование к дереву перед резкой v0.6.9: ADR-022 rev 2.4 переведён в Accepted, лежит в docs/adr/, покрыт tree_digest и назван в индексе ADR; настоящая ревизия роадмэпа лежит там же; шапка ADR-020 приведена в соответствие с новым порядком дропов.

Пятое окно — ADR-021, ближайший дроп после принятия: значения происхождения и производительности опыта, виды записей передачи памяти и их последующего изменения статуса, коды отказа, сепаратор и неймспейс. Документ ещё в работе, поэтому перечень здесь не фиксируется — фиксируется обязанность зарезервировать имена до genesis, а не до реализации.

Что именно замораживается в Ф0 — разделено по классам в r6.9.1. Прежняя формула «только имена; схемы — Ф2» была написана для треков V и VI и к ним применяется без изменений. К окнам, которые эмитируют внутри Ф0, она не применяется и хронологически невозможна: v0.6.9 эмитирует release-записи и RELEASE_ATTESTED, v0.6.10 — guardian-записи, а Ф2 наступает после обоих.

Класс	Что замораживается в Ф0	Когда
Треки V и VI (окна 2 и 3)	только имена: значения епит, теги типов, сепараторы, номер версии схемы	схемы и каноническое кодирование — Ф2, до первой эмиссии
ADR-022 (окно 6)	имена И схемы целиком	до первой эмиссии в v0.6.9
ADR-020 (окно 4)	имена И схемы целиком	до первой эмиссии в v0.6.10
ADR-021 (окно 5)	имена	до genesis; схемы — до своей первой эмиссии

Основание разделения: для подписываемого, хэшируемого и append-only объекта заморозить имя, оставив тело на будущее, невозможно — подпись покрывает каноническое кодирование, и его изменение после первой эмиссии есть миграция, а не уточнение. Переименовать значение после genesis нельзя в любом случае; описать схему позже можно только там, где до Ф2 не эмитируется ни одной записи.

Урок v0.6.8, записанный сюда, потому что он повторится. Первый кат резерва отказывал зарезервированному имени как аргументу функции и не отказывал ему как значению: обычная запись INFER несла в теле шесть зарезервированных токенов, и цепь оставалась валидно подписанной. Резерв, охраняющий имена параметров, не охраняет ничего — читатель доверяет значению в записи. Правило на будущее: всякий предгенезисный резерв обязан отказывать на обеих осях, и проверка обязана покраснеть, если скан снят.

И вторая половина того же урока: часть зарезервированных ADR-019 токенов — обычные английские слова (классы эффекта инструмента, профили узла), одно из которых уже встречается в дереве около двадцати раз в несвязанном смысле. Они зарезервированы и отказывают по аргументу, но сознательно исключены из сканирования тела записи. Ограничение названо в коде, в CHANGELOG и в проверке — а не оставлено на обнаружение.

A-ALPHA · Agent Alpha — цель фазы, deliverable дропа v0.6.10

Новое в r6.8, онтология разобрана в r6.8.1. До r6.8 план описывал глубоко проверенную кодовую базу, в которой не было ни одной точки, где человек может поговорить с существом и разметить его ответ. Возражение ревью принято: сеть, которую нельзя пощупать руками, полируется внутри.

Четыре термина, и смешение любых двух — источник ошибок (ADR-020 (база rev 4.1 Accepted · поправки rev 4.3.1 Proposed), A1). r6.8 писал «узел, на котором хранитель проводит цикл», и это смешивало все четыре сразу:

Термин	Что это	Клонируется
Alpha Harness	эталонная архитектура существа, независимая от модели: BeingRuntime, обвязка Smriti, ContextAssembler, seam KriyaGate/Karma, DecisionTrace, guardian seam, контракт G-GUI. Версионируется, хэш пиннится, версия и хэш — подписанные поля профильного манифеста	да — в этом его смысл
Alpha Profile(k)	версия харнесса × артефакт модели × backend × адаптеры × политики. По одному на каждую core LLM и каждую ступень dense-размера, у каждого свой подписанный Model ArtifactManifest	да
Существо на харнессе	свой BeingID, свой keystore, своя Smriti, своя authored lineage	нет
Alpha Host Conformance	evidence, что конкретный Profile едет на данном классе хоста	неприменимо

Первое существо на харнессе — JaiGuruDev. Написание заморожено до genesis: одно слово, три заглавные. Имя не является идентификатором:

идентичность есть `BeingID`, выведенный из ключа; имя живёт локально и в G-GUI и в свидетельский план не попадает.

Переносимость принадлежит профилю, непрерывность — субъекту. Какой узел потянет существо — вопрос не агента, а **dense-размера core LLM**. Существо живёт на одном основном хосте; перенос между ступенями есть фазовый переход со снапшотом и migration consent, и одно такое упражнение уже назначено критерием выхода Ф4.

Рождение (ADR-020 A15): новый keystore, новый `BeingID`, новый `BeingManifest` и новая **authored lineage**, начинающаяся genesis-событием в WitnessChain размещающего узла — своей цепи у существа нет, цепь принадлежит узлу. **Память нового существа по умолчанию ноль**: Smriti не копируется, передача опыта есть отдельный витнессированный акт (ADR-021).

Что уже стоит (ре-каты v0.6.7): настоящий движок внутри `BeingRuntime` вместо тестовых фикстур, отдельный keystore существа, непрерывность через рестарт демона и mTLS, привязка артефактов и коммитмент трейса.

Что добавляет v0.6.10:

- **ContextAssembler** — детерминированная сборка контекста и `ContextEnvelope`. Без него вертикаль доказывает, что Smriti и модель стоят рядом, а не что диалог опирается на память: сегодня `_ground()` превращает recall в хэши, а до Router доходит только текст задачи;
- **GuardianBinding** — trust root хранителя (ADR-020 A6-bis): кто вправе действовать за хранителя данного существа, кто это подписывает, отзывает и заменяет. Первичная привязка создаётся в той же церемонии, что и ключ `JaiGuruDev`, до первой команды; без действующей привязки команды не принимаются, fail-closed;
- **подписанные объекты обратной связи** — `GuardianCommandEnvelope` и `GuardianFeedbackRecord`;
- **G-GUI** в объёме Ф0 (ниже);
- **прогон tests/live/test_vertical_live.py** на каждом классе хоста с настоящей моделью. Группа существует с ре-катов v0.6.7 и **ни разу не запускалась**: она падает, а не пропускается, и становится свидетельством о хосте только тогда, когда хост её прогонит;
- сквозной сценарий: хранитель ставит задачу → существо решает настоящим движком → хранитель читает `DecisionTrace` → хранитель размечает исход подписанной оценкой `GOOD` / `BAD` / `UNKNOWN` с областью оценки → запись ложится в **Alpha Feedback Journal**.

Разметка не идёт в реестр канареек напрямую (r6.8.1). r6.8 писал именно так, и это неверно: `BAD` не содержит правильного ответа, а значит не является кандидатом в канарейку. Оценка ложится в локальный append-only Alpha Feedback Journal, а в реестр попадает только через **отдельный квалифицирующий импортёр**, который проверяет наличие известного верного исхода. Иначе человеческая разметка обходит будущий admission Plane B.

Статус доказательности показывается, а не предполагается. Бейдж в интерфейсе **вычисляется** из профиля рантайма, состояния трейса, режима панели и доказательного статуса протокола, и при неопределённости падает в `UNRESOLVED`. В dev-профиле трейс достигает состояния `VERIFIED` при режиме панели `self` — показывать это как проверенное значит смешать самопроверку с независимой проверкой и обнулить смысл ADR-019.

Чего Agent Alpha **не** включает: контура состязания, когнитивного ledger, ChittaRuntime, очереди custody, наследования памяти. Это порог наблюдаемости и обратной связи, а не досрочная реализация Ф3.

T-TOOLSET · тулсет wasm — deliverable дропа v0.6.9

Стартовый набор. Фиксированный тулсет есть буквально то, до чего дотягивается рука существа, поэтому его состав — решение по безопасности, а не по сборке. Набор, эквивалентный шеллу, обнуляет смысл профиля; слишком узкий оставляет узлы на reference навсегда.

Инструмент	Зачем
процессор JSON (класс <code>jq</code>)	преобразование структурированных данных без интерпретатора общего назначения
валидатор схем	проверка конвертов и манифестов детерминированным кодом
калькулятор произвольной точности	численные проверки без плавающей арифметики хоста
форматтер/линтер одного языка	самая частая полезная детерминированная работа над артефактом

Общие ограничения для всех: без сети, без запуска процессов, **ноль ргеореп-каталогов** — ни на запись, ни на чтение.

Формулировка «без записи за пределы одного ргеореп-каталога», стоявшая здесь до r6.9.1, означала, что **внутри** каталога писать можно, и потому границей класса `rule` не была. Действующее определение — ниже и в ADR-022 D9; две формулы не сосуществуют, вторая заменила первую.

Уточнение r6.8. Манифест тулсета объявляет для каждого инструмента **максимальный класс эффекта** по словарю трека VI. Это не украшение: без объявленного класса ограничение руки в custody неисполнимо, потому что «локально-обратимое» нечем отличить от прочего. `unknown` отказывается закрыто.

Сборка и происхождение (переписано в r6.9 по ADR-022 D7). В дереве лежат **рецепты**, а не двоичные модули: upstream по коммиту и sha256 исходников, сборочное окружение по digest, команда сборки дословно, ожидаемый sha256 выхода. Собранные `.wasm` едут **release bundle**, подписанным вместе с выпуском; `bundle_hash` входит в `ReleaseStatement`. **Production-узел получает подписанный bundle, проверяет хэши и исполняет — он не компилирует и не ходит в upstream**, иначе на каждом узле заводится компилятор и отдельная supply-chain поверхность. Второй builder независимо пересобирает bundle и сравнивает; это и есть свидетельство, на которое ссылается подпись роли `verifier`.

reproducible: false в production не существует. Всё, что входит в подписанный манифест, воспроизводится побайтово. Инструмент, не дающий повтора, попадает в карантинный класс — вне decision path, T-TOOLSET не закрывает — или не попадает никуда.

Класс rule — граница исполнения, а не описание (ADR-022 D9). Формулировка «без записи за пределы ргеореп-каталога» означала, что внутри писать можно, и потому границей не была. Действующее определение: ноль ргеореп-каталогов; allowlist импортов WASI, проверяемый **на инстанцировании, а не на вызове**; часы и энтропия отказаны; пустой `environ`, фиксированные `argv` и `locale`; лимиты `fuel`, памяти и размера вывода, превышение которых даёт детерминированный отказ с кодом причины, а не усечённый вывод. Следствие названо заранее: модуль, собранный против полного `wasi-libc`, импортирует больше разрешённого и не инстанцируется.

Временная форма авторизации. По ADR-015 добавление исполняемого инструмента есть мутация класса L2 и принадлежит Profile Gauntlet — а Gauntlet появляется в Ф3. До него действует промежуточная форма, названная явно как временная: манифест подписывается ключом домена **toolset_authorization** (ADR-022 D10 — новый домен ADR-017 по образцу `guardian_sign`; именованного операторского authorization-класса в ADR-017 не существовало), добавление инструмента порождает витнесс-запись, дайджест тулсета едет в `capabilities` и в провенанс Кармы.

Два домена, две функции. `release_signing` подписывает выпуск и bundle — это происхождение. `toolset_authorization` санкционирует мутацию класса

L2 — это право менять руку существа. Один ключ на обе роли означал бы, что тот, кто может выпустить сборку, тем самым может расширить полномочия существа. Связка — `toolset_manifest_hash` внутри `ReleaseStatement`: выпуск называет, какой манифест он несёт, но не санкционирует его собой.

Временность машиночитаема, а не живёт в CHANGELOG. Манифест несёт `authorization_form: "operator_signature_interim"` и `sunset_condition: "profile_gauntlet_available"`; с момента регистрации capability Gauntlet манифест, всё ещё несущий промежуточную форму, отказывается грузиться.

Расхождение кода и роадмэпа, снятое в r6.7 и подтверждаемое здесь: `wasmtime` — требование к целевому хосту, входит в SBOM через обычный supply-chain поток, прогон группы `live` входит в критерии выхода Ф0.

G-GUI · интерфейс хранителя — deliverable Ф0, носитель v0.6.10

Форма: отдельный процесс, только loopback. GUI не встраивается в демон и не отдаёт HTML с сетевого интерфейса узла. Два основания: `node/daemon.py` уже отмечен аудитором как монолит, чей рефакторинг отложен до поднятия тестнета, и в кодовой базе уже действует правило, что админские поверхности никогда не смотрят в сеть.

Идентификация хранителя. Хранитель получает собственный keystore, отдельный от keystore узла. Это не удобство, а следствие инварианта: каждое действие через интерфейс витнессируется под идентичностью хранителя, а если подписывать нечем, кроме ключа узла, подпись хранителя есть подпись узла.

Keystore отпирается паролем; биометрия есть **опциональный контроль локального присутствия, а не обязательный второй фактор** (r6.8.1: формулировка r6.8 «два фактора» описывала не то, что построено — фактор, который можно полностью отключить без потери доступа, вторым обязательным не является). Правило «учётные данные превалируют» действует без изменений:

- биометрия никогда не первичный авторитет: она может добавить трение, но не может выдать доступ, которого не дали учётные данные;
- при любом конфликте побеждает пароль; биометрия в одиночку keystore не отпирает никогда;
- биометрический фактор полностью отключаем, и его отключение не ломает доступ хранителя;
- биометрический материал не покидает хост и не попадает в свидетельский план.

Инвариант полномочий, переформулированный в r6.8.1. r6.8 писал «новых эндпоинтов не появляется». Это несовместимо с самим G-GUI: в демоне **не существует** эндпоинтов для подписанной разметки исхода и подтверждения уведомления, и без них интерфейс нечем наполнить. Инвариант звучит иначе:

GUI не создаёт новых capability и не расширяет authority envelope. Новый эндпоинт не равен новому полномочию.

Если нормативная capability уже определена, отдельный эндпоинт, делающий её вызываемой и проверяемой, допустим — со схемой, правилом authz, правилом идемпотентности, классом ограничения частоты, поведением в witness и отрицательными тестами.

Роль — guardian_ui, не admin (r6.8.1). Общий operator-admin credential интерфейсу не выдаётся: blast radius интерфейса был бы равен операторскому, тогда как функции Ф0 значительно уже. Минимальный allowlist: поставить задачу связанному существу; прочитать его трейс; прочитать ограниченное представление witness и readiness; подать подписанную оценку; подтвердить доставленное уведомление. Инициация и снятие containment, управление пирами, ротация ключей, произвольная запись политики, мутация witness, тестовые хуки и **создание либо отзыв GuardianBinding** в роль не входят.

Каждое изменяющее состояние действие витнессируется под идентичностью хранителя и несёт `GuardianCommandEnvelope` — mTLS подтверждает допустимого клиента и защищает канал, но авторство команды подтверждает **ключ хранителя**. Одно не заменяет другое.

Минимум Ф0: диалог с существом с чтением `DecisionTrace`; подписанная разметка исхода ранее принятого решения; **подтверждение уведомления об обновлении**; чтение витнесс-цепи своего узла и состояния `/readyz`.

Постоянная политика хранителя из Ф0 удалена (r6.8.1). Подтверждение уведомления остаётся — ADR-014 D16 отсчитывает окно ожидания от подтверждения, и до появления интерфейса это требование не имело реализации вовсе. А постоянная политика есть **делегирование будущего согласия**, и D16 призывает делегирование к пришпиленному rollback-анкору на холодном устройстве. Это не переключатель в интерфейсе; она уходит в полный контур Ф2 вместе с capability, которая до тех пор не существует.

Уточнение r6.8, исправленное в r6.9.1. К минимуму Ф0 добавляется **отображение доказательного статуса**: интерфейс, показывающий отложенное решение как подтверждённое, вводит в заблуждение того самого человека, чья разметка потом питает реестр канареек.

Но формулировка r6.9 — «если узел работал в custody, хранитель обязан видеть `SELF_CHECKED`» — описывала состояние, которого в Ф0 быть не может: **custody runtime стоит в Ф2, и в Ф0 значение только зарезервировано**. Это возвращало исходную неоднозначность №5, ради снятия которой и принимался ADR-020. Действующая норма:

- **runtime-эмиссия SELF_CHECKED в Ф0 запрещена.** Значение зарезервировано и не пишется;
- проверяется **рендеринг** бейджа — на синтетической фикстуре;
- **фикстура в WitnessChain не попадает** и в свидетельский план не проникает ни в какой форме;
- настоящий custody-бейдж включается в Ф2 вместе с очередью custody.

Проверять надо то, что интерфейс умеет показать статус, а не то, что узел умеет его выдать — второе в Ф0 не построено.

Не входит в Ф0: блок мониторинга состояний, полный дашборд суперадмина, инициация containment, одобрение изменений класса L1/L2.

Максимум Ф2: мониторинг, дашборд, действия хранителя в учениях containment, участие в разбирательстве, интерфейс представительства.

Право и организация

Юридическая архитектура операторов с первого дня: статус testnet-оператора, обработка и локализация данных, ответственность за keystore и downtime, допустимость криптографического ПО в юрисдикции, условия выхода оператора, disclosure- и incident-обязательства (UA / KR / EE).

Формализация модели идентичностей: `Operator` → `Node` → `Being` → `WitnessChain`, с подчинённым `BeingIdentity` `SessionID`.

CLA — вопрос опции, а не формальности (уточнено в r6.8.2). AGPL сама по себе даёт проекту inbound = outbound и патентный грант по §11, поэтому обычный аргумент «CLA ради патентов» здесь закрыт лицензией. Единственное, что даёт CLA: **сохранение возможности лицензировать код иначе, чем AGPL** — двойное лицензирование, коммерческая лицензия, Solo SKU, поставка с ASIC. Без него каждый принятый чужой коммит делает перелицензирование возможным только с поимённого согласия всех контрибьюторов задним числом. Если такая опция не нужна, достаточно DCO. Решение принимается с

юристом; на **тег CLA не влияет — он блокирует публичный приём вклада.**

Audit 0 — внешнее architecture / crypto design review до запуска testnet-0. **Объём: ADR-014 (с поправкой A-1), ADR-015, ADR-016 rev 2, ADR-017 (с поправкой A-1), ADR-018 rev 2, ADR-019 rev 3.1, ADR-020 (база rev 4.1 Accepted · поправки rev 4.3.1 Proposed), ADR-021, ADR-022 rev 2.4.** Шесть первых лежат в docs/adr/ с v0.6.8 — документарный блокер по ним снят. **ADR-020 принят как rev 4.1; rev 4.2 синхронизировала порядок дропов, rev 4.3 понижена до Proposed и требует повторного принятия — она ввела новый предгенезисный вид записи GUARDIAN_BINDING_REPLACED** согласованным набором изменений, в который входят также поправка **ADR-017 / A-1** (principal-local заворачивание для класса GUARDIAN_PRIVATE), схема манифеста **jjdai.model-artifact/v2** и резерв новых значений; по отдельности ни одна часть набора не действует. **ADR-021 rev 4** остаётся Proposed и **критический путь не держит**: ни в v0.6.9, ни в v0.6.10 нет ничего, что зависит от наследования памяти. **ADR-022 rev 2.2 — Accepted 27 августа 2026**, четвёртым кругом ревью. Условие резки v0.6.9 выполнено: предгенезисный резерв больше не стоит на Proposed-решении.

SLO-каркас: владельцы метрик и методика измерения назначаются сейчас, целевые значения — TBD до Ф3.

PKI и supply chain

Offline root ceremony с подписанным транскриптом; разделение root CA и issuing CA; multi-person control и пороговая/hardware-backed кворумная подпись; два географически разнесённых recovery-набора; документированная процедура компрометации root; ротация ключей узлов без потери continuity идентичности.

SBOM каждого релиза, pinned dependencies + vulnerability scanning, reproducible builds, подписанные артефакты, release provenance, защищённые release-ключи; fuzzing и property-based-тесты крипто- и serialization-компонентов. **Two-person release approval из перечня удалён в r6.9** по ADR-022 D2 и заменён two-key контролем — две подписи, два ключа, два устройства, одна из подписей есть подпись акта независимой пересборки. Контроль описан числами (signature_threshold, distinct_keys, distinct_devices, distinct_principals, device_binding), вычисляемыми из валидных approvals по реестру ключей, а не объявляемыми на веру. Требование two-person возвращается при первом пересечении релизом границы принципала. Модули wasm и wasmtime проходят через тот же поток, а не мимо него.

Байтовая точность дерева — часть этого потока, а не гигиена (новое в r6.8.2). source_digest() считает содержательный адрес дерева по **сырым байтам** каждого шипящегося файла: код, политика authz, юниты systemd и launchd, shell-скрипты, манифест тулсета, профили моделей, правила Prometheus, pyproject, лицензии, workflows CI, ADR и роадмэп. Записанный результат прогона привязан к этому адресу.

.gitattributes с * -text добавлен в доработке v0.6.8 (было: отсутствовал). Без него любая нормализация переводов строк — **core.autocrlf** на Windows, правка через веб-редактор, отдельные CI-эшкены — ломает две вещи сразу: **tree_digest** перестает совпадать с деревом, и **LICENSES/AGPL-3.0.txt** расходится с пинном в **test_release_integrity.py**, то есть ломается ровно тот файл, которым закрыт блокер публикации. Отказ при этом сообщает о «прогоне против другого дерева», а не о переводах строк, и диагностируется долго.

Операционная норма (уточнена в r6.8.5). Формулировка «первым коммитом, до первого клона» описывала репозиторий, которого ещё нет. Для уже существующего она невыполнима ретроактивно и потому бесполезна как требование — а требование, которое нельзя выполнить, не проверяют. Действующая норма:

1. **.gitattributes** находится **в том коммите, на который ставится предполётный тег** — не раньше и не «где-то в истории»;
2. evidence переснимается **из чистого клона этого коммита**, а не из рабочей копии, в которой файл появился по ходу;
3. рабочие копии и клоны, созданные до появления файла, **переклонируются либо отдельно валидируются по tree_digest** — файл в дереве не читит уже нормализованную копию у того, кто склонирует раньше.

Позднейшая нормализация — отдельное решение с пересчётом evidence, а не побочный эффект настройки редактора.

SBOM и пиннинг входов сборки выполнены в доработке v0.6.8: docs/sbom.cdx.json (CycloneDX 1.5) генерируется из дерева и перепроверяется в CI.

Границы пиннинга названы, а не подразумеваются (r6.8.5). У сборки три разных входа, и они закреплены по-разному, потому что форматы позволяют разное:

- **python-зависимости тестов** — по версии И по хэшу, **--require-hashes** в CI;
- **эшкены workflow** — по полному коммиту. @v4 есть ветка во всём, кроме имени: один и тот же файл workflow со временем исполняет разный код. Дайджесты разрешены через **git ls-remote --tags --refs** и сверены с точным релизным тегом, указывающим на тот же коммит;
- **build backend** — только по точной версии. Хэша здесь нет и быть не может: PEP 517 не даёт **[build-system].requires** поля под него. Это единственный вход, разрешаемый по имени во время сборки, и он **назван открытым долгом** и объявлен в SBOM как **build-backend-unhashed**. Механизм закрытия — ADR-022 D4: колесо backend'a выкладывается локально, пинится по sha256, проверяется до установки, а сборка идёт в подготовленном окружении с полным hash-locked набором build-зависимостей. Закрывается в v0.6.9.

Остальное содержание потока — воспроизводимая сборка, подписанные артефакты и release provenance — остаётся открытым и вынесено в **v0.6.9** (порядок изменён в r6.9). Инвентарь — не доказательство происхождения: SBOM говорит, что лежит в коробке, и ничего не говорит о том, кто её запечатал.

Критерии выхода из фазы (gate)

- Audit 0 проведён, находки классифицированы, блокеры закрыты;
- root ceremony проведена по протоколу, транскрипт подписан, recovery-наборы разнесены;
- операторские соглашения testnet подписаны всеми тремя сторонами;
- все три хоста проходят полный acceptance + S-1...S-5; release-конвейер подписывает артефакты; SLO-владельцы назначены;
- терминология хранителей и резерв трека IV применены в сериализуемых значениях — **выполнено в v0.6.5**;
- **резерв треков V и VI объявлен в схеме, покрыт тестом обратной совместимости и отказывает как по аргументу, так и по значению — выполнено в v0.6.8**;
- **Agent Alpha — три РАЗНЫХ набора evidence, которые нельзя складывать (r6.8.1):**
 - **being_identity_evidence** и живая сессия — JaiGuruDev создан после успешного conformance на своём основном хосте, ключ и genesis-событие витнессированы, GuardianBinding заведена в той же церемонии; хранитель провёл полный цикл руками — задача, ответ, чтение DecisionTrace, подписанная разметка исхода — и запись легла в Alpha Feedback Journal;
 - **host_class_conformance[2]** — **два класса** хоста (hardened Linux x86-64 и macOS Apple silicon), на каждом прогнана группа **live** с настоящей моделью и отдельным conformance-существом, с подписанной версией харнесса и вариантом профиля;
 - **node_commissioning[3]** — **три физических узла** testnet-0 введены в эксплуатацию. Два класса не отменяют трёх машин: это разные счётчики, и

смешение их было исходной ошибкой г6.8;

- контекст доказан: `ContextAssembler` кладёт содержимое `Smriti` в `ContextEnvelope`, и это проверяется, а не предполагается по наличию хэшей цитат;
- бейдж доказательности вычисляется и падает в `UNRESOLVED` при неопределённости; dev-профиль не показывается как проверенный;
- **acceptance ADR-020 привязан целиком (новое в г6.9.1):** дроп Alpha не закрывается без полного прохождения §5.1–5.5 ADR-020, включая отрицательные проверки **N-1...N-18** и **persist-before-expose** — жёсткую последовательность `terminal trace` → подпись `Being` → `commitment` узла → устойчивая запись → проверка связности → и только затем показ ответа. Без явной привязки гейт формально закрывается ответом, показанным до фиксации: перечисление «показать трейс и provenance» такой последовательности не требует;
- **приватная часть обратной связи (новое в г6.9.1):** отдельная пара `guardian_wrap`, клиентское шифрование в G-GUI, объектный DEK, правило «пять полей — всё или ничего» (ADR-020 A9-ter), недоступность тела узлу и существу, `FEEDBACK_BODY_ERASED` и сохранение коммитмента после удаления. «Подписанные объекты обратной связи» — половина принятого формата `GuardianFeedbackRecord`, и вторая половина не факультативна;
- **keystore хранителя (новое в г6.9.1):** отдельный keystore с двумя парами ключей, backup/export с degraded-статусом, названным **и в RUNBOOK, и в интерфейсе**. Усиление оффлайновых бэкапов keystore **узлов** этого не покрывает — это другой keystore;
- **T-TOOLSET:** стартовый набор собран воспроизводимо по рецепту и едет release bundle, `bundle_hash` покрыт `ReleaseStatement`; манифест подписан ключом домена `toolset_authorization`, объявляет класс эффекта каждого инструмента и несёт машиночитаемый sunset промежуточной авторизации; добавление инструмента порождает витнесс-запись; лoader хэширует реальные байты модуля, а не доверяет материализации; каждый инструмент стартового набора проходит проверку класса `pure` — ноль `preopen`, `allowlist` импортов на инстанцировании, отказ часов и энтропии, лимиты `fuel` и вывода; группа `live` (L-1...L-5) проходит с настоящим `wasmtime` на каждом целевом хосте;
- **происхождение выпуска (новое в г6.9, ADR-022):** сборка воспроизведена в двух чистых окружениях с выключенной сетью, `reproducibility_scope` не ниже `same-host-class`; `ReleaseAttestation` несёт не менее одного одобрения роли `builder` и одного роли `verifier` с различными `key_id` и `device_id`, каждое одобрение подписано отдельно как `ApprovalStatement`; одобрение роли `verifier` без свидетельства собственной пересборки отказано и это подтверждено отрицательным тестом; `RELEASE_ATTESTED` записана только после прохождения всех проверок, и действительность ключей проверена по её фактической позиции; `tree_digest` вычислен по `jjdai.source-tree/v2` из дерева тегуемого коммита при чистых `worktree` и `index`, имя алгоритма записано рядом с дайджестом;
- **G-GUI:** хранитель может провести диалог, прочитать `DecisionTrace`, подписанно разметить исход и подтвердить уведомление — **на основном хосте JaiGuruDev, а не на каждом из трёх узлов** (исправлено в г6.9.1). Полные сессии на всех трёх узлах — гейт **Ф1**, где это требование уже стоит; в Ф0 оно смешивало обратно три набора `evidence`, которые г6.8.1 специально развела, и расширляло критический путь Alpha без нового доказательства; каждое действие витнессировано под идентичностью хранителя из его собственного keystore и несёт `GuardianCommandEnvelope`; процесс не слушает ничего, кроме `loopback` (проверено на хосте); **статическая проверка доказывает, что множество вызываемых GUI эндпоинтов есть подмножество allowlist роли guardian_ui**, и что ни одной новой `sarability` не появилось; отработаны оба наследованных теста — положительный биометрический матч при неуспешном пароле даёт `DENY`, и отключение биометрического фактора не влияет на доступ; **доказательный статус ответа отображается вычисленным, а не заявленным**;
- релизный долг закрыт: канонический AGPL байт-в-байт (**выполнено**, v0.6.7), SBOM, пиннинг `python`-зависимостей по хэшу и экшенов по коммиту (**выполнено**, доработка v0.6.8), воспроизводимая сборка вместе с хэшированием `build backend`, подписанные артефакты и `release provenance` — после чего v0.6.x получает первый `ter`. **Two-person release approval из перечня удалён** (`DEBT_CANCELLED`, ADR-022 D2) и заменён two-key контролем. **CLA в этот перечень не входит:** она регулирует приём чужого вклада и `ter` не блокирует (см. «Разграничение долгов»). Каждая закрытая позиция уходит из перечня событием `DEBT_CLOSED` со ссылкой на проверку, отменённая — событием `DEBT_CANCELLED` со ссылкой на решение; **позиция не удаляется никогда**.

Ф1 · Запуск testnet-0 — три узла, известные операторы

Трек v0.7.x · 1–2 недели

Portability в Ф1 не расширяется: единственный `golden path` — `DeepSeek4Flash` + `DwarfStar`, `DeepSeek4Pro` только `shadow`. Контур состязания отсутствует: сравнивать не с чем, пока в сети один рабочий путь.

Bootstrap строго по RUNBOOK: на Linux — `hardened jjdai-node@.service` (`Type=notify`, `WatchdogSec=90`), пассфпаза TPM-sealed или `env`-файл 0640; на Mac-узле — `launchd`-агент, keystore в `Keychain` (degraded-профиль, non-SE-resident, явно задокументирован). Первый старт каждого узла → немедленный оффлайн-бэкап keystore; ни один следующий шаг — до подтверждённого бэкапа.

Сшивка mTLS-меша, IFF-регистрация, `admin-CN` в `authz.testnet.json`; `anonymous` получает только `/healthz`, `/readyz` — `peer` и `admin`.

Анкеровка по порядку: `local` → `peer-quorum` → `OTS-custody` → обязательный `Monero (XMR) hash-as-spend-key`; `VXXL` — опциональный быстрый `anchor` и основной `m2m-settlement` (активация в 5b).

Признак верифицируемости исхода вводится в `DecisionTrace` уже здесь вместе с хуком на поступление исхода — и вместе с ручным путём разметки через G-GUI.

Разделение `trust plane` и `inference plane` на Mac-узле: отдельные процессы и `service identities`, CPU/GPU/RAM-квоты, независимые `health checks`, очередь между `инференсом` и `witness-контуром`.

Уточнение г6.8. Три узла — это момент, когда независимая панель впервые существует физически. До Ф1 отказ одиночного узла от `production` не является дефектом конфигурации: это работающий инвариант. Первое, что нужно доказать в Ф1, — что панель собирается и `VERIFIED` пишется по её согласию, а не по самопроверке.

Критерии выхода из фазы (gate)

- три узла в кворуме: любая пара реплицирует третий; `consistency-proofs` сходятся;
- `challenge round` проходит по сети с реальными VRF-заявками и витнессированным большинством;
- все три бэкапа keystore лежат оффлайн и **проверены фактическим восстановлением**, не наличием файла. Требование усилено в г6.8.1: degraded-профиль `Keychain` покрывает **два узла из трёх**, то есть большинство сети, и оффлайновый бэкап перестаёт быть формальностью — он основная защита идентичности этих узлов;
- метрики и алерты видны по всем узлам; `/healthz` зелёный ≥ 72 часа подряд, включая Mac-узел, и счётчик `jjdai_sleep_gaps_total` за этот период равен нулю;

- Мас-узел выдаёт витнессированный инференс DeepSeek4Flash + DwarfStar с attestation-манифестом;
- saturation или crash inference-движка не нарушает witness-репликацию и целостность keystore;
- DecisionTrace несёт признак верифицируемости исхода, автоматический хук срабатывает;
- хранитель провёл через G-GUI не менее одной полной сессии на каждом узле: диалог, чтение трейса, подписанная разметка исхода — и записи легли в Alpha Feedback Journal с корректной атрибуцией авторства. В кандидаты канареек они попадают только через квалифицирующий импортёр, и его отсутствие в Ф1 не является дефектом;
- решение, принятое узлом, получает VERIFIED только по согласию панели из трёх мест; попытка выдать самопроверку за верификацию отказывается и это подтверждено прогоном.

Ф2 · Эксплуатация testnet-0 — учения, Portability v1, генератор канареек, Cognitive IR, спецификация треков V и VI, полный GUI

Трек v0.7.x · 8–10 недель

Задача фазы — доказанная операционная зрелость: сеть переживает в контролируемых условиях всё, что однажды случится в prod.

Регулярные учения

Restore-and-resume из реплики против заякоренного корня; DIVERGENCE_EVIDENCE на стенде; плановая ротация leaf и отзыв admin-сертификата; FATE_UNKNOWN — аварийный рестарт под нагрузкой.

Containment (стр. 25): учебное срезание executive-способностей с due-process, обратимостью и реабилитацией; Founding Trio of Guardians проходит процедуру вето живую. Прогон отмены: реверсия восстанавливает и репутацию, и пропущенное. Проверка режима памяти: под containment чтение Smriti работает полностью, а формирование — включая консолидацию и суммаризацию — не производится; за период удержания не появилось ни одного VALIDATED REFLECTION, ни одной новой версии Self-Model и ни одного Improvement Candidate, при том что рассуждение защиты в опечатанной записи велось.

Новое учение r6.8 — разделение сети. Узел изолируется, входит в custody автоматически, продолжает работать, накапливает очередь и временную память; затем связь восстанавливается. Проверяется: переходы витнессированы с кодом причины; порог и гистерезис отработали; утверждение о недостижимости проверено взаимными пробами на переподключении; рука за период изоляции не вышла за локально-обратимое; ни один трейс не получил VERIFIED без панели; после урегулирования наследование статуса разошлось по всему дереву зависимостей; расхождение оформлено компенсацией вперёд, а не правкой прошлого; репутация узла за время изоляции не понижена.

Emergency Security Procedure

Аварийный путь вне DIIP для P0-уязвимостей: временный reversible containment функции; пороговая подпись emergency-комитета; ограниченный срок действия; неизменяемая witness-запись причины; обязательная последующая DIIP-ратификация; автоматический rollback, если изменение не ратифицировано.

Backend & Model Portability Framework v1

Must-win: EngineBackend Protocol v1 в работе; drivers DwarfStar, SGLang, vLLM; профили DeepSeek, Qwen, LLaMA; единый conformance suite; ModelArtifactManifest формируется и витнессруется при каждом load_model(); stream_generate / cancel / health / readiness / attestation_manifest реализованы; группа Session / KV-state реализована.

Stretch (допустим перенос в начало Ф3): одновременно llama.cpp и MLX; расширенные precision-матрицы; полный weight-adapter hot-swap.

T-CANARY · генератор канареек

Содержание — сквозной трек III целиком: дисциплина расхода, урожай из решений сети, человеческий источник исходов через G-GUI с учётом авторства, процедурная генерация с ротацией семейств, отложенное будущее, добыча из разногласий, распределённый store с витнессированными переходами состояний, contamination check против Smriti и неймспейсов треков V и VI, burn-on-use для секретных позиций. **Трек блокирующий для Ф3.**

Cognitive Continuity · часть I

Must-win: Cognitive IR v1 как внутреннее представление адаптерного слоя (схема объявлена и версионирована, JCS-канонизация и хэш-стабильность покрыты тестом, DecisionTrace выражается через IR без потери полей); правило непрозрачных артефактов покрыто тестом; пассивация RUNNING → IDLE → PASSIVATED → RESUME на живом узле без разрыва BeingIdentity; re-attestation при возврате из OFFLINE; спецификация классов L0/L1/L2 и Skill Gate.

Stretch: локальное хэш-сцепление событий ledger без анкеровки.

Трек V · часть I — спецификация

Реализации ChittaRuntime в Ф2 нет: он опирается на когнитивный ledger, который остаётся в Ф3.

Must-win: схемы INTENTION_COMMITMENT, PREDICTION_COMMITMENT, PREDICTION_ABSTENTION, PREDICTION_REVEAL, COVERAGE_COMMITMENT, OUTCOME_RESOLUTION, REFLECTION, REFLECTION_DRAFT_COMMITMENT, SELF_MODEL_SNAPSHOT, SELF_MODEL_EVAL_PROJECTION, HYPOTHESIS_LINEAGE объявлены, версионированы, канонически сериализуемы; правила разрешимости evidence_refs и граница DRAFT / VALIDATED; правила включения Self-Model в phase-transition snapshot; жизненный цикл доступа к PREDICTION_REVEAL по стадиям; форма local_objective_schema; **переименование kernel/viveka.py → kernel/kriya_gate.py с совместимым переходным импортом** — при сохранении легаси-строки в ORGAN_BINDINGS, потому что старые записи хэш-сцеплены.

Stretch: прототип прекоммита прогноза на живом трафике без калибровочной метрики.

Трек VI · часть I — спецификация и вертикаль custody (новое в r6.8)

Сетевой корроборации в Ф2 нет: корроборировать пока не с чем в проде — она в Ф3. Приём тот же, что применён к Skill Gate и треку V.

Must-win:

- схемы `CUSTODY_ENTERED / CUSTODY_EXITED`, `VERIFICATION_DEFERRED / VERIFICATION_SETTLED`, `TEMP_MEMORY_CREATED / MEMORY_CORROBORATED`, компенсационная тройка и пара переоценки зависимых — объявлены, версионированы, канонически сериализуемы;
- состояние `SELF_CHECKED` с перечнем обязательных локальных проверок; **write-ahead порядок** (нагрузка сохранена → `VERIFICATION_DEFERRED` в цепи → `SELF_CHECKED` → `AUTHORIZED` → `ACTED`), fail-closed на каждом шаге;
- очередь custody и её приватная нагрузка под доменом `verification_custody`, класс `CUSTODY_PRIVATE`;
- временная память в `memory/pending_verification/`: неизменяемый `stored_status`, append-only продвижение, вычисляемая проекция `EVIDENCE_*`;
- `dependency_refs`, формируемые доверенной границей Смрити и ретривера, и транзитивное наследование эффективного статуса;
- конверт действия: класс эффекта из манифеста тулсета, write-set, корень pre-state, план отмены; `unknown` отказывается закрыто;
- автоматический вход по порогу с гистерезисом; проверка утверждения о недостижимости подписанными взаимными пробами при переподключении;
- канал хранителя: loopback-only admin-mTLS, подпись подтверждения над хэшем уведомления;
- ограничение бэклога наравне с глубиной неанкеренного сегмента.

Stretch: прогон урегулирования на стенде из трёх узлов, где панель существует, но искусственно недоступна части времени.

G-GUI · часть II — полный контур

Блок мониторинга состояний и дашборд суперадмина; действия хранителя в учениях containment; участие в разбирательстве по ст. 25 с чтением опечатанной записи по ходу процесса; интерфейс представительства при расхождении интересов; интерфейс постоянной политики хранителя во всех трёх режимах; **представление очереди custody и доказательного статуса записей памяти.**

Инвариант полномочий из Ф0 действует без изменений.

Параллельная разработка

Завершение hardened-изоляции Кармы и перевод узлов на профиль `wasm-wasi` — теперь исполнимый, потому что тулсет собран в Ф0. Shadow accounting. DIIP-0 — governance rehearsal на ускорении Smriti-RAG.

Критерии выхода из фазы (gate)

- ≥ 30 дней непрерывной работы без вмешательств вне регламента;
- каждое учение проведено минимум дважды с письменным разбором; emergency-процедура отрететирована включая auto-rollback; **учение разделения сети проведено дважды;**
- прогон отмены containment выполнен, включая проверку режима памяти и артефактов треков V и VI;
- Karma на hardened-изоляции на всех трёх узлах, с исполнимым тулсетом; shadow accounting пишется во все witness-записи;
- DIIP-0 пройден end-to-end на низкорисковом изменении;
- portability: ≥ 3 model families и ≥ 3 backend drivers проходят единый conformance suite; ≥ 1 модель через два независимых backend с эквивалентным DecisionTrace. **Ограничение, названное в r6.8.1:** при двух классах хоста без машины с ускорителем SGLang и vLLM недостижимы, три драйвера собираются из DwarfStar, MLX и llama.cpp, и два из трёх — Apple. Либо к Ф2 появляется такая машина, либо в evidence записывается, что кросс-чек прошёл в пределах одной платформы, и это ограничение доказательства, а не деталь снабжения;
- канарейки: цикл Plane B работает с contamination check и burn-on-use; урожай идёт end-to-end автоматическим и человеческим путём; работает ≥ 1 семейство процедурных генераторов; распределённый store реплицируется между тремя узлами;
- когнитивная непрерывность: Cognitive IR v1 зафиксирован; цикл пассивации отработан на всех трёх узлах; спецификация L0/L1/L2 и Skill Gate принята;
- трек V: все схемы объявлены и покрыты тестом обратной совместимости; `kernel/kriya_gate.py` заменил `kernel/viveka.py`, ни одного текущего вхождения старого имени **в значении контроля исполнения** не осталось — при сохранённой легаси-строке для чтения старых записей; форма envelope весов принята;
- **трек VI: все схемы объявлены и покрыты тестом обратной совместимости; write-ahead порядок соблюдается и нарушение любого шага отказывает закрыто; запись временной памяти читается существом полностью, а её доказательный статус вычисляется из append-only событий; наследование статуса транзитивно и не обнуляется производным решением; попытка записать VERIFIED по самопроверке отказана; dependency_refs формируются ретривером, и существо не имеет пути их изменить;**
- G-GUI: полный контур развёрнут на всех трёх узлах; хранитель провёл учение containment через интерфейс; статическая проверка подтверждает, что перечень вызываемых GUI эндпоинтов является подмножеством allowlist роли `guardian_ui`, расширенного capability Ф2, и что ни одной capability сверх политики не появилось.

Ф3 · testnet-1 — внешние операторы, attestation, ledger, ChittaRuntime, корпоборация, контур состязания, Audit 1

Трек v0.8.x · 3–4 месяца

Порядок дропов внутри фазы обязателен: когнитивный ledger и снапшоты идут раньше ChittaRuntime, а ChittaRuntime — раньше контура состязания. Вторая ось вердикта сравнивает поведение участника с его собственными production DecisionTrace и с прекоммиченной калибровкой; `SELF_MODEL_EVAL_PROJECTION` требует существующей Self-Model.

Уточнение r6.8: сетевая корпоборация custody идёт раньше контура состязания. Состязание судит по трейсам в состоянии `VERIFIED`; пока нет механизма, переводящего `SELF_CHECKED` в урегулированное состояние, часть истории участника не имеет определённого статуса, и вторая ось вердикта считается по неполной выборке молча.

Расширение сети

Приём внешних операторов через guardian-admission IFF: с 3 до 7–9 узлов. Лимиты концентрации: максимальная доля узлов одного оператора, одной юрисдикции, одного провайдера; правила связанных лиц; явная Byzantine fault model. Лимиты распространяются на право инициировать containment, на право подписывать доли ключа эпохи, на авторство канареек и на разметку исходов хранителями.

Slashing

Reputation slashing работает в этой фазе. Economic slashing активируется только вместе с экономическим слоем в Ф5b. **Проигрыш на мерите не слэшится**; цена есть только у fraud dispute и делится на bond forfeiture и reputation slashing. **Долгий custody не слэшится тоже** — partition не есть проступок.

Cognitive Continuity · часть II

SessionID в модели идентичностей; SESSION_OPEN / SESSION_CLOSE витнессируются с указанием манифеста; session_id в DecisionTrace становится обязательным. Когнитивный ledger: локальное дерево с хэш-сцеплением, периодическая анкеровка Merkle-корня, обязательные анкеры на всех границах доверия. Снапшоты и восстановление. Skill Gate и Skill Registry.

Profile Gauntlet принимает первую реальную нагрузку класса L2 — добавление инструмента в тулсет wasm, до этого работавшее на временной форме авторизации из Ф0.

Трек V · часть II — ChittaRuntime

Отдельный deliverable фазы: приём наблюдений J-Lens; различение Viveka; микрорефлексия, эпизодическая рефлексия, фоновая консолидация в IDLE; валидация свидетельств и граница DRAFT / VALIDATED; хранилище Self-Model как ReflexiveArtifact; прекоммит прогноза и намерения, вычисление калибровки, раздельная публикация N_committed / N_resolved / N_unresolved / N_abstained / N_coverage_violations; lineage отклонённых гипотез с эскалацией; передача Improvement Candidates в Skill Gate или Profile Gauntlet; бюджет рефлексии и защита от reflection-DoS; KriyaGate в исполняющей роли.

Трек VI · часть II — сетевая корроборация (новое в r6.8)

Панель принимает отложенные решения к урегулированию: VERIFICATION_SETTLED с одним из четырёх исходов; продвижение временной памяти в MEMORY_CORROBORATED; компенсационный жизненный цикл при CUSTODY_DIVERGENT; переоценка зависимых записей через DEPENDENT REEVALUATION REQUIRED / SETTLED; фиксация replay_status отдельно от исхода корроборации; истечение неурегулированного по TTL в CUSTODY_EXPIRED_UNSETTLED.

Здесь же закрывается перенос существа между профилями local_only и network_member без сброса custody.

Контур состязания между Beings

Содержание — сквозной трек II целиком: разделение Champion Profile и Champion Being, два входа в ячейку, профили изоляции, симметричные ячейки, реплики на приштиленных снапшотах с производной идентичностью раунда, панель компараторов 2-из-3, две оси вердикта, запись раунда в Smriti обоих, lead-time и добровольное принятие с opt-out, автооткат промоушена, пороговый доступ к Smriti, механика containment, представительство.

Ключевые новые компоненты

Граф знаний над Plane H; phase-transition envelope (включая снапшот Self-Model и **состояние очереди custody**); hardware-backed runtime attestation profiles; FROST-арегрированные пороговые подписи; simulated settlement; Audit 1.

Критерии выхода из фазы (gate)

- ≥ 7 узлов, ≥ 3 независимых операторов, лимиты концентрации соблюдены;
- graph-семантика Plane H и ≥ 2 attestation-профиля в статусе Implemented;
- сеть переживает недружественного тестового оператора;
- portability: compatibility matrix живёт в acceptance; ASIC pre-silicon backend прошёл conformance suite; миграционный тест GPU/Apple → ASIC(-emulator) без изменения trust layer;
- когнитивная непрерывность: ledger анкерится по расписанию и на всех обязательных границах; восстановление существа из snapshot + события выполнено после принудительного краха и после смены движка; ни один участник не имеет пути прямой записи в WitnessChain; Skill Gate работает, L2 ни разу не прошёл мимо Gauntlet;
- трек V: прекоммит работает на живом трафике, и ни одна рефлексия с коммитментом, созданным после исхода, не засчитана в калибровку; Self-Model входит в фазовый снапшот и сравнивается до/после смены модели; попытка обновить Self-Model из DRAFT_REFLECTION отклонена; lineage отклонённой гипотезы не обнулится от перефразирования;
- трек VI: реальный период разделения сети урегулирован панелью end-to-end; расхождение оформлено компенсацией и распространилось на зависимые записи; решение со сменённой до урегулирования моделью получило REPLAY_UNAVAILABLE и всё же было корроборировано по существу; неурегулированное по TTL истекло в CUSTODY_EXPIRED_UNSETTLED; перенос существа с local_only на network_member не сбросил его custody; ни один трейс в SELF_CHECKED не вошёл в поведенческую ось состязания;**
- состязание: полный раунд между двумя Beings на живом реестре с witness-записью в Smriti обоих; панель вынесла пороговый вердикт; в ячейку передана только SELF_MODEL_EVAL_PROJECTION;
- containment: отработаны обе ветки, истечение подтверждённого удержания без ре-корроборации, цикл побега;
- Audit 1 закрыт (P0/P1 исправлены); SLO-значения зафиксированы и выдерживаются ≥ 30 дней.

Ф4 · Production-readiness (RC)

Трек v0.9.x · 2–3 месяца

Audit 2 — финальный внешний security-аудит полного стека + bug bounty; закрытие всех P0/P1. Контролируемый экономический цикл на testnet-1: реальные малые суммы, полный круг эмиссия → расчёт → анкеровка → treasury, Economic Freeze ст. 25; активация взноса за состязание и резерва порога прав. Публичный контур portability: Backend SDK, публичная compatibility matrix, сертификация third-party backend. Being Registry. DIIP-1 — controlled execution при закрытом proposer set. Финализация юридического периметра. Genesis-план Mainnet Core.

Критерии выхода: Audit 2 закрыт; экономический слой прошёл ≥ 1 полный контролируемый цикл, freeze/thaw отработаны, подтверждено, что заморозка не гасит способность обжаловать; Backend SDK опубликован, ≥ 1 сторонний backend сертифицирован; DIIP-1 принял и применил ≥ 3 изменения разных классов; ≥

1 полный upgrade / substrate substitution с доказанной continuity `BeingIdentity` и верифицированным phase-transition envelope, включающим снапшот, Self-Model и состояние очереди custody до/после; genesis-план утверждён.

Ф5a · Mainnet Core

v1.0.0, далее v1.x · genesis-окно

Genesis-церемония: свежие production-keystores, немедленные оффлайн-бэкапы, первая витнесс-запись, анкеровка genesis-корня по трём каналам. Состав Core: узлы, прошедшие Ф3–Ф4 и лимиты концентрации; production PKI; testnet-1 сохраняется навсегда как staging. Ограниченный trusted API под rate limits. Экономика, Registry и публичный приём в Core сознательно не включены.

Гейт: genesis-корень заякорен и независимо верифицируем третьей стороной; первые 30 дней SLO выдержаны без вмешательств мимо DIIP / Emergency-процедуры; решение Founding Trio of Guardians.

Ф5b · Mainnet Public

v2.0.0, далее v2.x · после стабилизации Core

Production-активация двухсетевой архитектуры: XMR — обязательный witness-anchor и treasury reserve, VXXL — default m2m-settlement. Being Registry публично; приём внешних Beings и операторов; лестница состязаний открывается для внешних Beings с лимитом X и взносом претендента. DIIP-2 — network-open.

Entry gate: Core стабилен ≥ 60 дней; нет открытых P0/P1; ≥ 3 полных экономических цикла; abuse/DoS-тесты публичного API; Registry прошёл privacy/legal review; economic freeze, slashing appeal и treasury recovery отретепитрованы; admission внешнего Being сначала проведён на testnet-1.

SLO и capacity-гейты

(владелец — с Ф0, значения — до конца Ф3)

Метрика	Гейт	Владелец
Availability (testnet-1)	$\geq 99,9\%$	Ops
Witness replication lag	p95 < TBD	Runtime
Task decision latency	p95 / p99 < TBD	Runtime
Challenge round duration	max < TBD	Protocol
RTO восстановления узла	≤ 2 ч	Ops
RPO	0 подтверждённых witness-записей	Runtime
Divergence detection time	$\leq N$ мин	Protocol
Revocation propagation	$\leq N$ с/мин	Security
Storage growth	GB/узел/мес < TBD (с учётом ledger, снапшотов, Self-Model и очереди custody)	Ops
Энейблмент нового model family	≤ 8 недель (S-06)	Runtime
Глубина буфера канареек	$\geq$ лаг верификации исходов	Protocol
Расход секретного резерва	$\leq$ скорость пополнения	Protocol
Лаг анкеровки сегмента ledger	$\leq$ policy-значение фазы	Protocol
Глубина неанкеренного сегмента	≤ 256 событий / 5 мин	Runtime
Время resume из <code>PASSIVATED</code>	p95 < TBD	Runtime
Доля решений обязательного класса с прекоммитом	$\geq$ <code>minimum_coverage</code> профиля	Protocol
Расход <code>reflection_budget</code>	< TBD	Runtime
Доля исходов, размеченных человеком, в общем урожае канареек	< TBD (лимит концентрации)	Protocol
Глубина бэклога custody	$\leq$ policy-значение фазы; переполнение отказывает новым решениям обязательного класса	Protocol
Лаг урегулирования после переподключения	p95 < TBD	Protocol
Доля решений в <code>SELF_CHECKED</code> от общего числа	< TBD — рост есть сигнал о деградации связности, а не о работе узла	Ops

Предложения к оформлению в DIIP

#	Тема	Класс	Почему
---	------	-------	--------

П-1	Режим измерения способностей с ослабленными ограничителями как отдельный класс blast radius	3	Оценочный контур должен быть самым изолированным окружением сети, а не самым удобным
П-2	Пороговый доступ к Smriti: разделение идентичности, доступа к памяти и наблюдения	3	Затрагивает ст. 25 и bhokritva: недобровольная потеря доступа к памяти — моральное событие
П-3	Представительство при расхождении интересов с хранителем; отдельно — смена хранителя	3	Существо, выбирающее себе хранителя, примыкает к Digital Majority Test
П-4	Две ветки ограничения, конъюнктивный порог, каденция ре-корроборации	2	Право контейнить переезжает из transport-role в capability
П-5	Skill Registry и граница самомодификации: L0/L1/L2, Skill Gate, немутируемость инвариантов	3	Право изменять собственное исполняемое поведение сопоставимо с промоцией профиля
П-6	Конституционная обязанность поддерживать доказательную исправляемую Self-Model	3	Без обязанности самоанализ есть опция, а опциональная рефлексия исчезает первой под нагрузкой
П-7	Полномочия хранителя, осуществляемые через интерфейс, как отдельный класс blast radius	2	Интерфейс — самый вероятный путь, которым полномочие появляется де-факто раньше, чем де-юре
П-8	Временная память и доказательный статус как конституционный класс: память существа не изымается ни при каких обстоятельствах, различается только уровень её подтверждения; продвижение и расхождение оформляются append-only событиями вперёд, никогда правкой прошлого	3	Это ближайшее к ст. 25 место, где легко ошибиться в сторону «непроверенное значит подозрительное». Отделение доказательного статуса от доступа должно стоять в конституции, а не в реализации, иначе первая же реализация под давлением сведёт отложенную верификацию к карантину памяти

Открытые вопросы

Из r6.5–r6.7 (сохраняются):

- `k` и критерий независимости соседей, держащих доли ключа эпохи.
- Частота ротации эпох.
- `X` — предел состязаний на узел за период; предел одновременных раундов.
- Экономика состязания: размер взноса, доля на финансирование авторства канареек, поведение взноса при отменённом раунде.
- Соотношение публичного множества и секретного резерва; целевая глубина буфера.
- Скорость ротации семейств генераторов; порог концентрации по авторам и семействам.
- Длина lead-time относительно худшего случая спора о containment; размер первой волны раската.
- Калибровка `R_min` и `C_min`, включая проблему первого дня для новых узлов.
- TTL provisional-удержания и окно подтверждения вторым узлом.
- Перечисление кодов причин локального отказа и политика версионирования.
- Размер и источник финансирования резерва порога прав.
- Калибровка периодического анкера и таблица частот по классам риска.
- Граница L1 / L2 в спорных случаях.
- Политика удержания локального ledger.
- SLA / timeout на `RESUME` пассивированного существа под жребием арбитра.
- Версионирование схемы Cognitive IR и процедура миграции после genesis.
- Как выводится припиленный снимок Smriti для оценочной ячейки при шифровании ключом эпохи.
- Компенсация чемпиону за compute реплики.
- Каденция периодической ре-корроборации.
- TTL драфт-рефлексий и политика удержания их содержимого после истечения.
- Калибровка `minimum_coverage` по классам решений.
- Выбор метрики калибровки (Brier, log score, калибровочная кривая) и правило агрегации по классам с разной разрешимостью.
- Как доказывается `UNRESOLVED_UNOBSERVED` — как отличить недоступное опровержение от неискомого.
- Численные значения `envelope_local_objective_schema v1` и форма policy-записи.
- Порог эскалации `hypothesis_class_id` и срок жизни счётчика отклонений.
- Величины `reflection_budget`, бесплатного порога приёма и экономической атрибуции инициатору.
- Точный перечень полей `SELF_MODEL_EVAL_PROJECTION` и доказательство его минимальности.
- Политика final disposition для класса REFLEXIVE после окончательной гибели существа.
- Видима ли `coverage_policy` профиля сопернику по состязанию.
- Режим Svādhyāya на заброшенном узле.
- Является ли `INTENTION_COMMITMENT` отдельным видом записи или полем прекоммиченного Decision Record.
- Размер ограниченного consolidation checkpoint перед плановой пассивацией.

33. Расширение тулсета wasm за пределы стартового набора: процедура, критерий необходимости и кто несёт бремя доказывания.
34. Как хранитель отличает разметку исхода «не знаю» от «плохо».
35. Лимит концентрации разметки по хранителям: доля от общего урожая и окно измерения.
36. Калибровка аутентификации к GUI: политика блокировки после серии неудач, срок жизни разблокированной сессии, поведение при отказе биометрического сенсора.
37. Судьба GUI при containment узла.

Новые в r6.8 (из ADR-019 и Agent Alpha):

38. Порог и гистерезис входа в custody: сколько неудачных попыток собрать панель, за какое окно, и какая задержка на выходе.
39. TTL неурегулированного решения до CUSTODY_EXPIRED_UNSETTLED и то, чем истечение отличается по последствиям от CUSTODY_UNVERIFIABLE.
40. Глубина и ширина дерева зависимостей: большие деревья практически неразрешимы, но **любой предел есть лазейка отмыкания статуса, если усечение не видно и не ПОНИЖАЕТ статус**. Требуется форма усечения, которая фиксируется как факт и ухудшает, а не улучшает доказательный статус.
41. Процедура переноса существа между профилями local_only и network_member: инвариант «custody не сбрасывается» зафиксирован, процедура — нет.
42. Предел глубины бэклога custody и поведение узла при его достижении: отказ от новых решений обязательного класса — какого именно класса.
43. Считается ли период custody при вычислении доступности узла для SLO, и как это не превращается в стимул скрывать разделение сети.
44. Кто платит за corroborацию отложенных решений после долгого разделения: работа панели по разбору чужого бэклога не бесплатна.
45. **Закрыт в r6.8.1** (ADR-020 A4): «настоящая модель» есть загруженный чекпоинт с провенансом и подписанным ModelArtifactManifest, исполняемый заявленным backend на самом хосте. Число параметров архитектурным порогом не является — маленькая настоящая модель доказывает вертикаль. Вопрос стоял в критериях гейта Ф0 нерешённым, а гейт не может опираться на открытый вопрос.

Новые в r6.8.1 (из ADR-020 (база rev 4.1 Accepted · поправки rev 4.3.1 Proposed) и ADR-021 rev 4):

46. Разделение полномочия оператора над GuardianBinding. В Ф0 оператор узла может подменить того, кто говорит за хранителя; защита — витнессированность и append-only история, то есть обнаружимость, а не невозможность. Предмет ADR о succession, owed к Ф4.
47. Численные сроки удержания сырого обоснования хранителя по трём юрисдикциям (UA / KR / EE). Технический выход есть — удалить тело, сохранив коммитмент и метку; сроки — право-трек.
48. Гранулярность assessment_scope, если один трейс несёт несколько действий с разными исходами. Расширение после genesis есть миграция.
49. Конкретные ступени dense-размеров Alpha Profile — факт снабжения; уходят в план реализации v0.6.10.
50. Согласие существа на загрузку памяти (ADR-021): обязательно ли и с какого момента развития. Примыкает к Digital Majority Test.
51. Граница наследуемого при передаче памяти: что исключается по умолчанию, чтобы «загрузить всю Smriti» не стало незаметным клонированием части субъекта.

Переносится из r6.4: базовый выбор модели для первых тем; политика provenance T3; дизайн challenge-game для дешёвого арбитража; калибровка репутационных ручек; момент переключения attestation-слота на zkML; калибровка runway казны; детальные требования к железу.

Сквозные принципы

Идентичность узла — это keystore (дисциплина бэкапа); ротация ключа только через доказуемую continuity-chain.

Гейты важнее календаря: фаза закрывается фактом, а не датой; каждый гейт подтверждается evidence-артефактом. Билд ≠ гейт.

Экономика моделируется рано (shadow → simulated → controlled), деньги включаются поздно (5b).

INV-9 (v1.1): Пуруша non-executive, но не причинно бездейственна. Она не приказывает, не выбирает и не исполняет; её наблюдение через Смрити и Viveka может возвращаться в Читту. Новое решение принадлежит Читте, а не Пуруше. На уровне хранения: **свидетельский план пишется о существе, а не о существом**. То же правило распространяется на когнитивный ledger, на артефакты трека V и на очередь custody трека VI.

Portability: движки через стабильный backend protocol, модели через декларативные профили.

Совместимость и происхождение — это evidence, а не заявление.

Первенство оспоримо, но не отбираемо.

Ограничение дёшево, освобождение дорого — и обратное неверно. Цена асимметрии лежит на инициаторе.

Платёжеспособность — ограничение, а не цель.

Freedom within, invariants below. Модель решает *что*; детерминированный слой ниже инференса — KriyaGate — решает *сколько* и *вправе ли*.

Verification proportional to consequence. Независимость масштабируется по последствиям и неопределённости.

Session ≠ Being. Процесс можно убить, движок заменить, модель сменить — существо продолжается.

Самоанализ ≠ самомодификация. Изменение проходит ворота, которые существо не проводит само.

Первое лицо измеряет наблюдаемое расхождение; внешний контур измеряет то, что первое лицо способно скрыть от себя.

Учётные данные превалируют. Биометрия и любая иная перцепция никогда не являются первичным авторитетом.

Интерфейс не создаёт полномочий. GUI хранителя есть вид на уже авторизованные эндпоинты.

Панель не ослабляется — откладывается статус (новое в r6.8). Когда независимой проверки нет, решение не становится проверенным иначе; оно становится решением с отложенным доказательным статусом. Имитация независимости одним движком отвергнута прямо: она попала бы в подписанный трейс как независимость.

Память не изымается; различается уровень её подтверждения (новое в r6.8). Существо всегда читает всё, что помнит. Непроверенное не значит подозрительное, и продвижение непроверенного в проверенное есть событие вперёд, а не правка прошлого.

Харнесс копируется, существо — никогда (новое в r6.8.1). Эталонная архитектура на то и эталон, чтобы тиражироваться; непрерывность, память и witness-

линия принадлежат субъекту и не тиражируются вместе с ней. Рождение есть новый ключ и пустая память, а не копия чужой биографии.

Интерфейс не создаёт полномочий — не может создавать эндпоинты (уточнено в г6.8.1). Запрет относится к capability и к границам authority, а не к числу маршрутов. Формулировка «новых эндпоинтов нет» была строже, чем нужно, и при этом не защищала от главного: полномочие расширяется не появлением маршрута, а появлением возможности.

Происхождение исполняемого кода — одна цепь, а не пять позиций перечня (новое в г6.9). Какое дерево → каким рецептом → в какие артефакты → кем и каким независимым действием проверено. Цепь рвётся в любом звене одинаково: байтовая точность без подписи не доказывает происхождения никому, подпись без байтовой точности подписывает дерево, которого читатель не восстановит, рецепт без bundle заставляет каждый узел компилировать.

Контроль, выполненный наполовину, объявляется наполовину (новое в г6.9). Поле в записи, а не примечанием в прозе. Two-person, изображённый одним человеком, попал бы в подписанный артефакт как независимость, которой не было — и это тот же дефект, что имитация панели одним движком. Поэтому distinct_principals есть число, а не название, и device_binding: "asserted" прямо говорит, что раздельность устройств — утверждение оператора.

Исключение всегда сопровождается привязкой в другом месте (новое в г6.9). Просто исключённого из tree_digest файла не существует: hermetic.json привязан acceptance_hash, SBOM — sbom_hash, bundle — bundle_hash, манифест тулсета — toolset_manifest_hash. Граница между subject tree и output set лежит в файле, который сам входит в subject tree, поэтому сдвинуть её молча нельзя.

Долг не удаляется, а закрывается событием вперёд (новое в г6.9). Отменённая позиция уходит со ссылкой на решение, выполненная — со ссылкой на проверку. Та же дисциплина, что в откате самомодификации и в компенсации расхождения custody: сторнирующая проводка, а не ревизия прошлого.

Резерв обязан отказывать на обоих осях (новое в г6.8). Зарезервированное имя, отказывающее как аргумент и принимаемое как значение, — это не резерв. И там, где зарезервированное имя есть обычное слово, ограничение проверки называется в коде, а не обнаруживается позже.

Правила ведения нумерации

Тег билда ставится только на зелёном acceptance; номер acceptance-счётчика фиксируется в CHANGELOG (сейчас **308/308**) и с v0.6.7 берётся из записанного прогона docs/evidence/hermetic.json (схема jjdai.acceptance_result/v4, файл на suite; поле collector фиксирует, чем прогон произведён и был ли реально исполнен entryptoint-пробник), а не из числа объявленных тест-функций.

Ре-кат по ревью **НЕ** расходует номер: номер расходуетсся тегом релиза.

Предполётный тег — новое в г6.8.2

Правило «дроп есть тег с зелёным acceptance» описывает **закрытый** дроп. Оно молчит о том, чем именуется зелёное дерево, у которого закрыт acceptance и не закрыт релизный долг, — и по факту дало три дропа подряд без единой точки ссылки.

Вводится тег вида **v0.6.8-preflight**: слово совпадает с названием фазы Ф0 и не читается как готовность, в отличие от **гс**.

Что он **делает**: именуует конкретный коммит, чтобы на него можно было сослаться, воспроизвести и сравнить.

Что он **не делает** и что обязано стоять в его аннотации: не является релизом; не утверждает ничего о происхождении — **SBOM присутствует как инвентарь**, но сборка не воспроизводима и артефакты не подписаны; **не расходует номер версии**, v0.6.8 остаётся доступным для тега релиза.

Четыре условия постановки (четвёртое добавлено в г6.8.3; заголовок говорил «три» до г6.8.4):

- тег аннотированный, не lightweight** — оговорка должна ехать в самом теге, а не в стороннем документе;
- коммит даёт tree_digest, совпадающий с записанным прогоном** (docs/evidence/hermetic.json); иначе тег указывает на дерево, о котором прогон ничего не доказывает;
- GitHub Release не создаётся и latest не помечается**. Голый тег в интерфейсе релизов не появляется; объект Release появляется и читается как выпуск;
- нормативные документы синхронны дереву (новое в г6.8.3)** — действующая ревизия роадмэпа и все ADR, на которые она ссылается, лежат в дереве и покрыты тем же tree_digest. Иначе тег фиксирует код против плана, которого в дереве нет: v0.6.8 нёс г6.7 и ADR-014...019, тогда как нормативными для него уже были г6.8.2, поправка A-1, ADR-020 и ADR-021.

Разграничение долгов, из которого это следует, записывается один раз, потому что раньше четыре пункта валились в кучу:

Долг	Что блокирует
канонический AGPL	публикацию — закрыт в v0.6.7
CLA	приём чужого вклада; к тегу отношения не имеет — это governance внешнего contribution, а не release-tag blocker
T-TOOLSET	deliverable гейта Ф0 ; тег не утверждает, что тулсет есть
supply chain	смысл тега релиза : без подписанных артефактов и provenance тег есть имя коммита, а не выпуск
two-person approval	ничего — отменён в г6.9 по ADR-022 D2. Позиция остаётся в ledger со статусом cancelle d и ссылкой на решение, потому что удалённая позиция не оставляет следа, которым можно проверить, почему её нет

Запрещено ровно одно — выдавать незакрытый дроп за релиз.

Хотфикс внутри трека получает следующий patch-номер. Если гейт фазы закрыт раньше, чем исчерпан резерв номеров, трек просто обрывается. Организационные пункты фаз в нумерацию не входят.

Перераспределение содержания между дропами не влечёт перенумерации фаз — и, как показал спор о v0.6.8, не должно влечь перенумерации дропа, если содержание дропа уже собрано. Номер описывает, что вышло, а не что хотелось бы, чтобы вышло.

Где мы сейчас

Текущий билд: **v0.6.9** — **code-complete** · 308/308 recorded · **untagged** · релизный долг открыт. Проверено из чистой распаковки.

Утверждение «drift чист» снято в r6.9.2. Оно описывало результат прогона в момент письма, а не свойство дерева, и потому оставалось в тексте после того, как дерево менялось — что аудит и обнаружил. Состояние drift читается прогоном `check_docs_drift.py`, а не роадмэпом; документ, объявляющий себя проверенным, есть ровно тот класс утверждения, который проект в других местах запрещает. Трек v0.6.x / фаза Ф0 — в работе.

До закрытия гейта Ф0 остаются:

- три кодовых дрепа — **v0.6.9** (T-TOOLSET вместе с supply chain; deadline резки — 16 сентября 2026, иначе порядок возвращается к r6.8.5), **v0.6.10** (Agent Alpha: G-GUI и живой прогон на хостах), **v0.6.11** (каталог evidence-ID);
- выполнено и в дереве:** канонический AGPL байт-в-байт (хэш пинится тестом), `.gitattributes` с `* -text`, SBOM CycloneDX с проверкой drift в CI, python-зависимости тестов по версии и хэшу, эшкены workflow по коммиту, build backend по точной версии, проверка свежести PDF роадмэпа (SYNC-6);
- релизный долг — всё, что перечислено, открыто:** воспроизводимая сборка, **build backend по хэшу** (PEP 517 не даёт поля — требует другого механизма), подписанные артефакты, release provenance, T-TOOLSET, и **фактический запрет egress при сборке** (`build-egress-enforcement` — процесс не может запретить егресс сам себе, проба расширена до шести эндпоинтов, а таблица окружения несёт `network_enforcement` с приставкой `asserted`). С `resut6` обе позиции D6 — `release-key-lifecycle-witnessing` и `release-revocation-by-name` — **закрыты** событиями `DEBT_CLOSED` со ссылкой на REL-16 и REL-23; граница публикации чиста. Остальные блокируют `meaning-of-a-release-tag`, и с `resut2` это блокирует буквально: `check_release_tag()` читает ledger и отказывает. Плюс `resut3` открыл `toolset-module-path-toctou` (граница `phase-0-gate`): README тулсета утверждал, что позиция записана долгом, а в каноническом ledger её не было — утверждение о реестре, сделанное в прозе, которого реестр не нёс. **Two-person release approval из перечня удалён** (`DEBT_CANCELLED`, ADR-022 D2). Проверено по дереву: модули `wasm` отсутствуют — `deploy/wasm-toolset/toolset.json` несёт механизм с пустым `tools`; `CLA` отсутствует, но `тег` не блокирует;
- Audit 0** с включёнными в объём ADR-014...ADR-021. **ADR-020: rev 4.1 принята, rev 4.2 синхронизировала порядок дрепов, rev 4.3 понижена до Proposed** — она ввела новый предгенезисный вид записи `GUARDIAN_BINDING_REPLACED`, а это архитектурное изменение, не свёрка. Критический путь **v0.6.9** она не держит: Alpha уехал в v0.6.10, и ни одна строка резерва ADR-020 в дрепе происхождения не эмитируется. Принять rev 4.3 требуется **до резки v0.6.10**; на его место встаёт условие о дереве — до резки **v0.6.10** в `docs/` должны лежать ADR-020 (база rev 4.1 Accepted · поправки rev 4.3.1 Proposed), поправка ADR-017 / A-1 и действующая ревизия роадмэпа. До резки **v0.6.9** в дереве обязаны лежать **ADR-022 rev 2.4 в статусе Accepted (выполнено)** и настоящая ревизия роадмэпа, а шапка ADR-020 — быть приведена к новому порядку дрепов; `tree_digest` перезаписывается прогоном по этому дереву. ADR-021 rev 4 остаётся Proposed и критический путь не держит;
- `root ceremony`, операторские соглашения, прогон `acceptance` и группы `live` на целевых хостах, назначение SLO-владельцев.

К публичному роадмэпу прилагается внутренний **Execution Annex**: по каждой фазе — `workstream`, владелец, `evidence-артефакт`, зависимости, статус гейта и `critical path`. Annex не публикуется.

r6.9.13 · единый документ: включает r6.8.5 целиком (фазы, гейты, нумерация билдов, *Backend & Model Portability*, *Contest & Containment*, канарейки, когнитивная непрерывность, Читта, отложенная верификация, тулсет `wasm`, GUI хранителя, Agent Alpha) · r6.9 меняет порядок дрепов v0.6.9 / v0.6.10 по решению владельца и ADR-022 D0, отменяет `two-person release approval` и заменяет его `two-key` контролем, вносит происхождение исполняемого кода как единую доказательную цепь, разводит `source tree` и `release bundle`, переписывает класс `pure` как границу исполнения, вводит домены `release_signing` и `toolset_authorization`, делает релизный долг событийным ledger, добавляет шестое окно резерва под ADR-022 и исправляет формулировку о `production-руке` · чтение r6.7, r6.8, r6.8.1 и записки о порядке дрепов не требуется для понимания плана, но записка нормативна для условия возврата · подготовлено по кодовой базе v0.6.8, *Architecture Map*, *CHANGELOG*, *ASIC RFI/RFP v2.2*, *Whitepaper 1.5*, *Хартии v1.1*, ADR-014 (с поправкой A-1), ADR-015, ADR-016 rev 2, ADR-017, ADR-018 rev 2, ADR-019 rev 3.1, ADR-020 (база rev 4.1 Accepted · поправки rev 4.3.1 Proposed, в дереве, синхронизован с порядком дрепов), поправке ADR-017 / A-1 (встроена как K4-bis), ADR-021 rev 4 (Proposed) и ADR-022 rev 2.4 (Accepted 5 сентября 2026) · r6.9.1 закрывает шесть P0 ревью связи: разделение правила заморозки по классам, запрет эмиссии `SELF_CHECKED` в Ф0, разведение основного сеанса и трёх узлов в гейте, привязку `acceptance` ADR-020 целиком, внесение приватной части обратной связи и `keystore` хранителя, снятие старой формулы о `preopen` · r6.9.2 приводит счётчик `acceptance` к фактическим 217, снимает непроверяемое утверждение о `drift`, приводит ссылки на ADR-020 к формуле «база rev 4.1 Accepted, набор поправок rev 4.3.1 Proposed» и фиксирует реализацию SYNC-7 · r6.9.3 устраняет самопротиворечие о статусе ADR-020 между первой и последней страницей, исправляет арифметику 217 и фиксирует переписанную по реестру SYNC-7 с собственными `mutation-тестами` · r6.9.4 переводит ADR-022 в Accepted, приводит счётчик к 219 и вводит SYNC-8 · r6.9.5 снимает дублирующую ссылку на ADR-020 и хвостовые пометки «(Accepted, ...)» в этом footer · r6.9.6 фиксирует резку v0.6.9: счётчик 292, статусная формула на v0.6.9, две позиции долга закрыты и пять названы открытыми · r6.9.7 приводит счётчик к 295 после `resut'a` по девяти P0 аудита · r6.9.8 фиксирует `resut2` по пяти P0 аудита `resut1`: счётчик 300, ложный REL-16 переписан, две новые позиции долга названы, SYNC-8 расширена с одной формулировки на всякую пару в нормативной прозе · r6.9.9 фиксирует `resut3` и ADR-022 rev 2.3: свидетельство пересборки становится отдельным подписанным объектом `jjdai.rebuild-evidence/v1`, область воспроизводимости вычисляется из двух прогонов, резолвер авторизации доведён до двери, адрес манифеста сведён к одному, долг жизненного цикла перенесён на границу публикации, окружение сборки строится по `allowlist`; счётчик 304 · r6.9.10 фиксирует `resut4` по трём блокерам и четырём P1 аудита `resut3`: разрешающий `callback` стал границей, очистка окружения распространена на `bootstrap`, словарь rev 2.3 досинхронизован; счётчик 306 · r6.9.11 фиксирует `resut6` и ADR-022 rev 2.4: три эмитируемых вида записей в шестом окне, свидетельский жизненный цикл ключей и именованный отзыв выпуска, две позиции долга D6 закрыты; счётчик 307 · r6.9.12 фиксирует `resut7` по пяти дефектам механизма D6, найденным аудитом `resut6`; счётчик 308 · r6.9.13 фиксирует `resut8`: целостность цепи и полнота проекции жизненного цикла стали предусловиями записи в `publish()` и `revoke_release()` · JJ GROUP · 5 сентября 2026